淮 安 大 学

课程设计报告

学　　院：	计算机与软件工程学院
专　　业：	数据科学与大数据技术
设计（论文）题目：	基于多路召回与实时行为信号的电影推荐系统设计与实现
学 生 姓 名：	韦余圳（组长）
	王文华
	杜伟
	王宇漩、刘康

2026
年 10
月

中 文 摘 要

本报告在课程冻结数据包（8,459 部电影、5,000 位用户、798,775 条评分）上实现了一套可复现的电影推荐系统：在 Starter Kit 基础上完成 Popular、Content、ItemCF、ALS 与冷启动五路召回和统一离线评价，并扩展加权 RRF 融合、Kafka 实时行为与短期兴趣重排、受控工具问答助手和兴趣自适应融合权重。评价遵循课程冻结协议（验证集 4,209 人、测试集 4,198 人，正反馈阈值 4.0，宏平均 Precision@10 与 Recall@10）：ItemCF 最优（0.0553/0.0978），ALS 次之（0.0414/0.0672），无历史用户热门榜达 0.3369。

关键词：推荐系统；协同过滤；矩阵分解；离线评价；实时行为；大语言模型

外 文 摘 要

Title: Design and Implementation of a Movie Recommender System with Multi-source Recall and Real-time Behaviour Signals

This report presents a reproducible movie recommender system built on the course frozen dataset of 8,459 movies, 5,000 historical users and 798,775 ratings. On top of the provided Starter Kit we implemented five recall algorithms (Popular, Content, ItemCF, ALS and Cold-Start), a unified offline evaluation pipeline, and four extensions: weighted Reciprocal Rank Fusion, Kafka-based real-time behaviour with short-term interest re-ranking, a natural-language assistant restricted to whitelisted read-only tools, and interest-adaptive fusion weights. Algorithms are shipped as model artifacts with manifests; training runs on the host while online inference uses the standard library only, and the business and evaluation databases stay isolated. Evaluation follows the frozen protocol (validation 4,209 users, test 4,198 users, positive feedback at rating >= 4.0, macro-averaged Precision@10 and Recall@10, ties broken by ascending movie id). ItemCF performs best (0.0553 / 0.0978 on validation), followed by ALS (0.0414 / 0.0672); the popularity baseline reaches 0.3369 on the no-history partition, and raising the ALS candidate support floor to 500 improves validation Precision@10 from 0.0054 to 0.0416. The running system passes 31 integration checks, four assistant scenarios, four behaviour scenarios and three adaptive-weight comparisons, with page-level screenshots as evidence.

Keywords: Recommender System; Collaborative Filtering; Matrix Factorization; Offline Evaluation; Real-time Behaviour; Large Language Model

目 录

1 绪论 1
1.1 研究背景与意义 1
1.1.1 方法与事实说明 1
1.1.2 证据与分析 2
1.2 国内外研究与应用现状 3
1.2.1 方法与事实说明 3
1.2.2 证据与分析 4
1.3 主要研究内容 5
1.3.1 方法与事实说明 5
1.3.2 证据与分析 5
1.4 论文组织结构 6
1.4.1 方法与事实说明 6
1.4.2 证据与分析 7
2 相关理论与关键技术 7
2.1 推荐系统基本原理 7
2.1.1 方法与事实说明 7
2.1.2 证据与分析 8
2.2 基于内容的推荐 8
2.2.1 方法与事实说明 8
2.2.2 证据与分析 9
2.3 协同过滤与矩阵分解 9
2.3.1 方法与事实说明 9
2.3.2 证据与分析 10
2.4 系统开发关键技术 11
2.4.1 方法与事实说明 11
2.4.2 证据与分析 12
3 系统需求分析与总体设计 12
3.1 系统需求分析 12
3.1.1 方法与事实说明 12
3.1.2 证据与分析 13
3.2 系统总体架构设计 13
3.2.1 方法与事实说明 13
3.2.2 证据与分析 15
3.3 系统功能模块设计 15
3.3.1 方法与事实说明 15
3.3.2 证据与分析 17
3.4 数据库设计 17
3.4.1 方法与事实说明 17
3.4.2 证据与分析 18

3.5 推荐服务接口设计 18
3.5.1 方法与事实说明 18
3.5.2 证据与分析 20
4 推荐算法设计与实现 20
4.1 数据集与预处理 20
4.1.1 方法与事实说明 20
4.1.2 证据与分析 21
4.2 Popular 热门推荐 22
4.2.1 方法与事实说明 22
4.2.2 证据与分析 23
4.3 Content 基于内容的推荐 23
4.3.1 方法与事实说明 23
4.3.2 证据与分析 24
4.4 ItemCF 协同过滤 25
4.4.1 方法与事实说明 25
4.4.2 证据与分析 26
4.5 ALS 矩阵分解 26
4.5.1 方法与事实说明 26
4.5.2 证据与分析 28
4.6 新用户冷启动 28
4.6.1 方法与事实说明 28
4.6.2 证据与分析 29
4.7 扩展模块设计 30
4.7.1 方法与事实说明 30
4.7.2 证据与分析 32
5 系统实现与功能测试 33
5.1 系统运行环境 33
5.1.1 方法与事实说明 33
5.1.2 证据与分析 34
5.2 用户与电影业务实现 35
5.2.1 方法与事实说明 35
5.2.2 证据与分析 36
5.3 推荐算法实验室实现 36
5.3.1 方法与事实说明 36
5.3.2 证据与分析 38
5.4 推荐接口与系统集成 40
5.4.1 方法与事实说明 40
5.4.2 证据与分析 41
5.5 系统功能测试 41
5.5.1 方法与事实说明 41
5.5.2 证据与分析 42
5.6 扩展功能实现 43
5.6.1 方法与事实说明 43
5.6.2 证据与分析 44
6 推荐算法实验与结果分析 46

6.1 实验环境与数据设置 46
6.1.1 方法与事实说明 46
6.1.2 证据与分析 46
6.2 评价指标 47
6.2.1 方法与事实说明 47
6.2.2 证据与分析 47
6.3 基础算法对比实验 48
6.3.1 方法与事实说明 48
6.3.2 证据与分析 48
6.4 不同用户的推荐结果分析 49
6.4.1 方法与事实说明 49
6.4.2 证据与分析 49
6.5 关键参数影响分析 50
6.5.1 方法与事实说明 50
6.5.2 证据与分析 51
6.6 实验结果讨论 53
6.6.1 方法与事实说明 53
6.6.2 证据与分析 53
总结与展望 54
7.1 工作总结 54
7.2 不足与展望 55
参考文献 56
附录 A 小组成员分工与贡献 57
附录 B 仓库与提交记录 58
附录 C 补充实验与测试记录 58
附录 D AI 辅助开发说明 58

1 绪论

1.1 研究背景与意义

1.1.1 方法与事实说明

本项目面向的是电影信息过载条件下的选择困难问题。课堂提供的冻结数据包包含 8,459 部电影与 5,000 位历史用户的 798,775 条真实评分，单部电影的可用信息主要是标题、上映年份与类型标签，用户侧信息是历史评分与收藏行为。对普通观众而言，候选集合规模远超个人可浏览范围，且电影之间缺少可以直接比较的统一尺度，用户不得不在“凭印象挑片”和“反复翻页”之间付出时间成本。本小组要解决的问题因此可以具体化为三点：在候选集合不小于 8,000 部的前提下把结果收敛到 10 条；让排序依据对用户可解释；让“新用户没有行为”和“老用户行为稀疏”这两种常见情况都有明确、可复现的处理方式。

系统价值分为课程训练价值与实际业务价值两层。课程训练价值是把推荐系统的完整链路跑通，包括数据冻结与版本校验、五类召回算法的实现、离线评价协议的执行、在真实容器里的前后端集成，以及在评分全部来自真实用户行为的前提下做参数对照实验。实际业务价值是让一个没有推荐背景的使用者能够直接获得可解释的推荐结果：每条结果都带有来源算法、模型版本、召回名次与匹配度，用户评分或收藏之后下一次推荐会发生可观测的变化。为避免把一般性优点写成已验证结论，本节只把“接口返回、离线指标、场景脚本结果”三类可复查的证据视为成果，其余一律按设计目标表述。

研究边界也需要提前说明。数据包中的导演、主演、时长、语言、简介与海报字段全部为空，因此本项目的特征工程只建立在类型标签与真实交互之上，不做虚假的属性补全；电影元数据的缺失被记录为数据限制而不是被忽略。评价侧严格使用课程冻结的划分与候选规则，不允许在测试集上反复调参。扩展部分只实现手册中真实完成的内容，未完成的模块（例如联网重建课程的离线镜像、公网部署）在报告中明确标注为未做。

- 问题场景：8,459 部候选电影与个人浏览能力的落差；新用户零行为、老用户行为稀疏。
- 目标用户：课程环境中的注册账号（可写行为）与只读历史用户（冻结行为）。
- 系统价值：可解释的 Top-10 推荐 + 可复现的离线评价 + 真实运行的扩展模块。
- 研究边界：仅使用冻结数据与真实行为；不补造元数据；不把未来计划写成已完成成果。

1.1.2 证据与分析

下图给出系统在真实浏览器中的首页运行状态。页面顶部是登录后的用户标识，中间是“为你推荐”的 Top-10 卡片，每张卡片显示片名、类型、名次、来源算法标签与匹配度；右侧是冷启动入口与历史用户演示入口。该截图来自本机容器（http://127.0.0.1:8080），不是设计稿，其中的电影、类型与分数都来自数据库与模型工件。

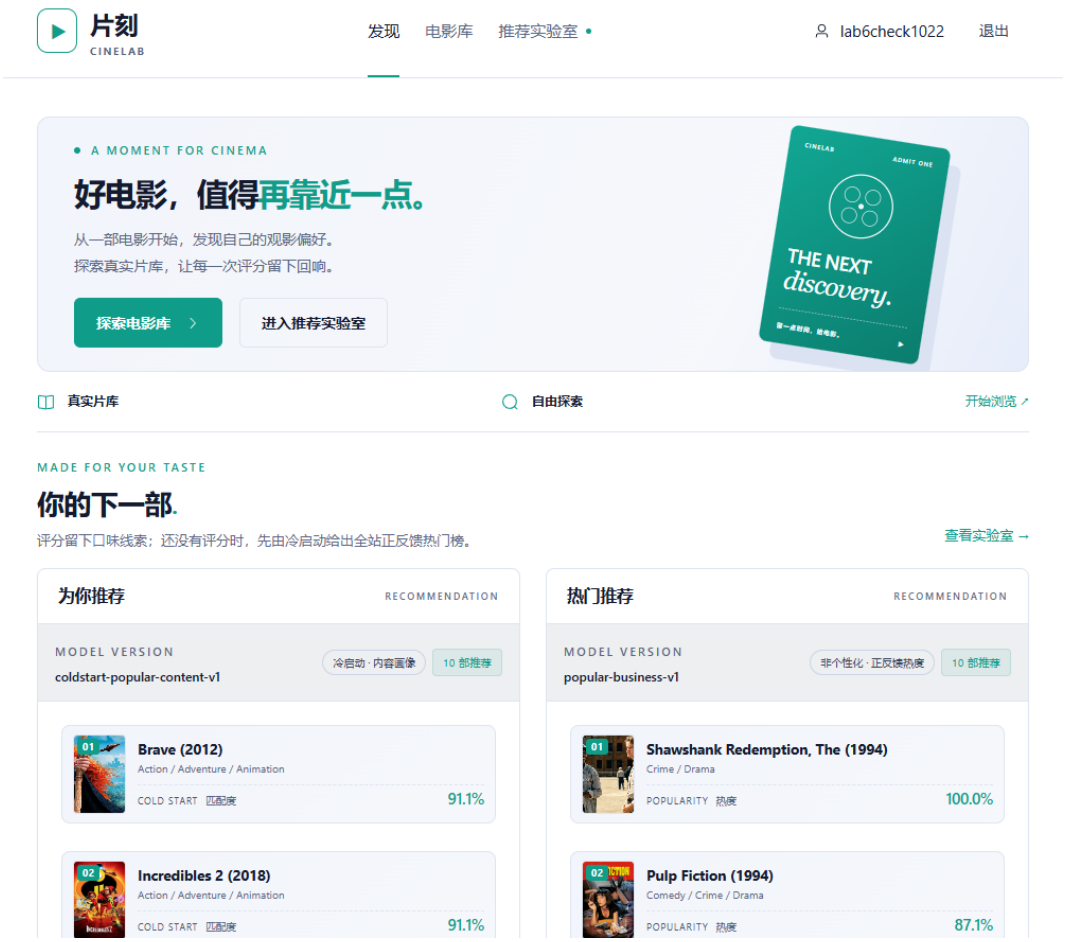


图 1-1 注册账号首页的推荐结果（真实运行）

从运行结果可以看到三点与设计目标一致的事实。第一，推荐列表对已交互内容是封闭的：接口返回的 10 条结果与当前账号已评分、已收藏的电影集合交集恒为空，

这一点在集成检查脚本中以程序方式断言。第二，结果可解释：每条结果都带有 model_version 与 recall_sources，用户可以从“为什么推荐这部”追溯到具体算法。第三，行为会改变结果：账号在给《玩具总动员》打出 5.0 分之后，下一次请求的画像权重发生变化，Top-10 列表随之改变，变化可以在接口响应中直接观察到。

仍存在两点限制。一是电影元数据缺失导致冷启动阶段的“兴趣选择”无法按导演、演员等维度展开，只能以类型标签为入口；二是推荐结果的解释停留在“来源 + 分数”层面，尚未提供自然语言层面的逐条理由，这一点由第 4.7 节的扩展模块以受控方式部分弥补。

1.2 国内外研究与应用现状

1.2.1 方法与事实说明

推荐系统的主流方法可以按信息来源分为四类。第一类是基于内容的推荐，它把物品表示为特征向量，把用户表示为历史偏好物品特征的加权聚合，再以相似度排序，优点是物品侧信息即可工作、可解释性强，缺点是对特征质量依赖大，容易出现“越推越窄”的同质化结果。第二类是协同过滤，其中基于物品的协同过滤（ItemCF）从共现行为构造物品之间的相似关系，能够捕捉“看过 A 的人往往也看 B”这类无法由属性表达的联系，缺点是受交互稀疏性影响，冷门物品难以获得足够共现。第三类是矩阵分解，以隐向量内积拟合评分矩阵，代表性工作是 Netflix Prize 之后的 ALS 与 SVD 系列方法，它在稠密交互上表达能力更强，但对训练数据规模、超参数与冷启动的处理要求更高。第四类是融合与重排，把多路召回的名次合并（例如 Reciprocal Rank Fusion），或在业务信号（时间、上下文、实时行为）驱动下对召回结果二次排序。

每类方法的具体使用条件在本项目中都被真实约束过。内容推荐使用类型 TF-IDF 向量与余弦相似度，评分权重取 $\max(\text{rating} - 3.0, 0.1)$，4.0 分对应权重 1.0、5.0 分对应 2.0，避免低分行为被当成同等强度的偏好。ItemCF 在二值化正反馈（$\text{rating} \geq 4.0$）上计算余弦相似度，并设置最小物品支持度 20、最小共现次数 5、每部电影保留 Top-50 邻居，以抑制长尾噪声。ALS 使用 Spark MLlib 的显式评分设置（implicitPrefs = false），rank = 32、regParam = 0.05、maxIter = 15、随机种子 20261022，并对不在因子表中的用户做评分加权的因子回填（fold-in）。融合采用加权 RRF，只在名次层面合并，权重由验证集上的对照实验确定。

文献与资料方面，本项目参考了矩阵分解的经典总结、ItemCF 与 Top-N 评价的

原始工作、RRF 的提出文献、基于内容的推荐综述、MovieLens 数据集论文，以及中文推荐系统与统计学习教材；工程实现部分参考了 Vue 3、FastAPI、MySQL 8、Apache Kafka 与 Spark MLlib 的官方文档。选择这些方法的理由有两条：第一，课程数据只提供类型标签与交互，四类方法恰好覆盖“特征、共现、隐向量、融合”四种信息通道，便于在同一协议下对比；第二，五类算法都可以在容器内以纯 Python 或 Spark 本地模式运行，不需要外部服务，结果可复现。

表 1-1 1.2 国内外研究与应用现状

项目	记录 1	记录 2	记录 3	记录 4
方法类别	基于内容	ItemCF	矩阵分解	融合/重排
代表工作	Pazzani & Billsus (2007)；Lops 等 (2011)	Sarwar 等 (2001)	Koren 等 (2009)；Hu 等 (2008)	Cormack 等 (2009)
核心思想	物品特征向量与用户画像的相似度	共现行为上的物品相似度	评分矩阵的低秩隐向量分解	多路名次合并与业务信号重排
适用条件	物品特征可用、可解释性要求高	交互足够、共现可统计	交互较稠密、可承受训练成本	已有多路召回、需要统一排序
本项目采用	类型 TF-IDF + 余弦	余弦相似度 + Top-50 邻居	Spark MLlib ALS（rank 32）	加权 RRF + 兴趣重排
主要局限	同质化、特征缺失时退化	稀疏物品无邻居	冷启动用户需回填	权重需实验确定

1.2.2 证据与分析

把四类方法放到同一冻结协议下对比之后，可以得到与本项目数据规模相符的结论：ItemCF 在验证集上取得最好的 Precision@10（0.0553），ALS 次之（0.0414），Popular 为 0.0342，Content 与 ColdStart 为 0.0041。这个排序与“本数据集的交互密度更高”这一事实一致——训练集 583,502 条评分覆盖 4,500 位用户与 8,459 部电影，物品共现关系比类型标签携带了更多区分信息。同时，Content 在冷启动分区（k=0）之外的增量也说明纯特征方法并非无效，只是在本协议的相关物品集合上命中率较低。

图表说明了“方法选择必须以本数据集上的证据为准”这一判断：文献中常见的结论（矩阵分解通常优于邻域方法）在本数据集上并未成立，原因在于本项目使用的是 Top-10 相关物品命中率而非评分误差指标，且物品侧的共现信号非常集中。限制同样明显：单次划分只覆盖 validation 与 test 两个分区，算法之间的差距在小数第三位上，尚未做多折交叉验证；因此报告只陈述“在本协议、本数据版本下”的相对关系。

1.3 主要研究内容

1.3.1 方法与事实说明

本小组的工作分为四条主线。第一条是数据处理与冻结校验：读取课程数据包，校验 8,459 部电影目录与 manifest 摘要，确认训练、验证、测试、冷启动四个分区的行数与有效用户数，并把 business 与 evaluation 两套数据严格分离。第二条是算法实现：在 Starter Kit 预留的 recommender/student/ 目录下实现 Popular、Content、ItemCF、ALS、ColdStart 五个推荐器，为每个推荐器生成模型工件与清单（manifest），并通过课程的契约校验（check_model、check_result）。第三条是业务集成：把算法接入后端接口与前端推荐实验室，补充离线评价表、融合实验入口、自然语言助手与行为事件面板。第四条是效果评价与验证：执行统一评价协议，输出验证集与测试集指标，做权重、隐向量维度、支持度下限等对照实验，并用集成检查脚本与场景脚本把“真实运行”固化为可复查证据。

- 继承功能：认证与 CSRF、电影目录与详情、评分收藏、只读历史用户演示、MySQL 八表结构与初始化、Nginx 静态站点、前端基础页面。
- 自主修改：五个推荐器与模型工件、离线评价脚本与前端评价表、加权 RRF 融合与权重对照、Kafka 行为事件与短期兴趣重排、LLM 受控工具助手、海报缓存与全站浅色主题、排布与留白优化。
- 运行证据：reports/ 下的评价结果、融合对照、场景脚本报告、集成检查报告，以及 docs/ 下的验收截图。
- 对应章节：算法见第 4 章，集成与测试见第 5 章，实验与参数分析见第 6 章，扩展模块见 4.7 节与 5.6 节。

1.3.2 证据与分析

下面给出小组自主实现的文件范围。算法侧集中在 recommender/student/（content.py、popular.py、itemcf.py、als.py、cold_start.py、rrf.py、evaluation_data.py、evaluation_entry.py）；工件侧在 models/business/ 与 models/evaluation/ 下的五个模型目录；脚本侧在 scripts/（模型构建、离线评价、指标发布、场景与集成检查）；后端扩展在 backend/app/（fusion.py、interest.py、behavior.py、behavior_api.py、llm.py、assistant.py、posters.py）；前端改造在 frontend/src/（AlgorithmPanel、FusionPanel、AssistantPanel、各页面与主题样式）。所有文件都能在仓库的六次主提交中定位。

表 1-2 1.3 主要研究内容

项目	记录 1	记录 2	记录 3	记录 4
自主实现文件	recommender/student/ 8 个模块	backend/app/ 7 个扩展模块	frontend/src/ 组件与页面	scripts/ 10 个脚本
模型工件	content-v1	popular-v1	itemcf-v1	als-v1（business/evaluation 各一套）
接口新增	/experiments/fusion/sources	/me/recommendations/fusion	/me/assistant/query	/me/behavior/interest
验证脚本	lab6_integration_check.py（31 项）	lab6_assistant_scenarios.py（4 场景）	lab6_adaptive_check.py（9 项）	lab5_scenarios.py（4 场景）

该表的用途是让评审能够按文件与脚本定位每一项声明。未列入本表的课程既有能力（例如认证、CSRF、电影列表）在报告中一律标注为继承功能；扩展模块中未实现的部分（公网部署、课程离线镜像的联网重建）在 7.2 节明确说明。

1.4 论文组织结构

1.4.1 方法与事实说明

全文按“背景与理论—需求与设计—算法实现—系统实现—实验分析—总结”的顺序组织。第 1 章说明问题、现状、工作内容与论文结构；第 2 章给出推荐系统的基本概念与本项目采用的四类方法原理，并交代开发关键技术的实际版本与职责；第 3 章从需求出发给出总体架构、功能模块、数据库结构与接口设计；第 4 章逐一说明五类召回算法与两个扩展模块的设计与实现，并给出可核对的参数与工件；第 5 章记录运行环境、业务实现、实验室实现、接口集成与测试方法；第 6 章给出统一评价协议、指标定义、基础算法对比、两类用户的结果分析、参数影响分析与结果讨论；最后总结工作、说明不足与展望，并在附录中列出分工、仓库与提交记录、补充实验记录与 AI 辅助开发说明。

- 第 1 章与第 2 章：问题定义、方法原理与关键技术选型依据。
- 第 3 章：需求、架构、模块、数据库与接口设计，对应实现前的设计决策。

- 第 4 章：五类算法与扩展模块的输入、公式、参数、过滤与回退规则。
- 第 5 章与第 6 章：真实运行证据与实验结果，形成“设计—证据—分析”的闭环。

1.4.2 证据与分析

为保证章节之间的对应关系可核对，本报告采用统一的编号约定：图号与表号按章编排，正文引用图号与表号；算法实现中的关键参数同时出现在第 4 章（实现说明）与第 6 章（参数实验）；提交记录统一放在附录 B，正文只引用提交主题之下的核对结论。完成全文后再核对章节标题与目录的一致性，避免出现模板中的占位标题。

本节的自查结论：全文结论均由仓库中的运行结果支撑，凡未实际运行的部分（联网重建离线镜像、公网部署、大模型推荐解释）均标注为未完成或未实现，不以计划代替结果。

2 相关理论与关键技术

2.1 推荐系统基本原理

2.1.1 方法与事实说明

推荐系统解决的问题可以形式化为：给定用户集合 U、物品集合 I 与交互记录集合 R，为每位用户 u 生成一个长度受限的排序列表。本项目的物品集合是数据包中的 8,459 部电影，交互集合是 5,000 位历史用户的 798,775 条评分（1.0—5.0，步长 0.5）以及注册账号的评分与收藏。与评分预测任务不同，本系统面向 Top-N 推荐任务：只需要命中用户真正认可的物品，而不要求预测出精确分数，因此评价指标使用 Precision@10 与 Recall@10，而不是 RMSE。

在 Top-N 任务中，常用指标有 Precision@K、Recall@K、F1@K、MAP 与 NDCG@K。Precision@K 衡量前 K 条中相关物品的比例，Recall@K 衡量前 K 条覆盖全部相关物品的比例，两者都依赖“什么算相关”的判定。本项目遵循课程冻结协议：正反馈定义为评分不低于 4.0（$\text{rating} \geq 4.0$），相关物品集合限定在“当前用户候选目录”之内，即目录中该用户尚未交互过的电影。这样定义把“已看过”与“是否喜欢”分开，避免把用户评过分的电影再次算作命中。

工程实现上，推荐链路通常分为召回与排序两段：召回阶段用代价较低的方法从全量目录筛选候选，排序阶段再用精细模型或业务规则打分。本项目五类算法都能在全量目录上直接给出 Top-10（目录规模 8,459 部），但仍按召回范式组织：每路算

法返回带名次的候选，融合模块在名次层面合并，短期兴趣模块再对融合结果做一次可解释重排。每个环节都对应具体文件与命令，概念不停留在术语层面。

- 输入数据：8,459 部电影的目录与类型标签；5,000 位历史用户的 798,775 条评分；注册账号的评分与收藏。
- 召回：Popular、Content、ItemCF、ALS、ColdStart 五路；排序：融合（名次合并）与短期兴趣重排。
- 过滤：所有算法一律剔除当前用户已评分与已收藏的电影，集成检查以程序断言该性质。
- 输出：Top-10 列表，每条带算法、模型版本、分数、召回来源与名次，便于解释与复现。

2.1.2 证据与分析

指标与候选定义的细节会直接影响结论，因此本项目用集成检查脚本把关键定义固化：接口返回的推荐结果与当前账号已评分、已收藏电影的交集必须为空；同一响应内电影不重复；rank 从 1 连续编号；同分时按 movie_id 数值升序排列。这些断言在每次回归检查中重新执行，任何一处被破坏都会使检查失败。

小节结论：本项目的评价体系沿用课程冻结协议（固定用户集合、固定候选规则、宏平均、Top-10 分母固定为 10），代价是无法用更宽松的指标美化结果，收益是报告中的每个数字都能被第三方用同一套命令复现。

2.2 基于内容的推荐

2.2.1 方法与事实说明

基于内容的推荐把物品表示成特征向量，再计算物品与用户画像之间的相似度。文本类物品常用 TF-IDF 或词向量表示：TF-IDF 以词频与逆文档频率的乘积衡量词的区分能力，对短文本与标签类特征尤其适用；词向量（word2vec、BERT 等）能表达语义相近但字面不同的特征，但需要较大语料与额外依赖。本项目的物品特征只有类型标签（genres），每部电影平均 2—3 个标签，因此选择稀疏、可解释、无外部依赖的 TF-IDF 表示。

具体实现分三步。第一步构造物品向量：统计每个类型标签在目录中的出现次数，计算逆文档频率 idf = ln(总电影数 / 含该类型的电影数) + 1，对每部电影的类型取 idf 值得到稀疏向量并做 L2 归一化。第二步构造用户画像：取该用户历史正反馈电影（$\text{rating} \geq 4.0$）的向量加权平均，权重为 $\max(\text{rating} - 3.0, 0.1)$，使 4.0 分权重为 1.0、

5.0 分为 2.0，3.0 分及以下按 0.1 的极小权重处理，避免低分行为被当成强偏好。第三步排序：计算画像与候选电影向量的余弦相似度，按分数降序、同分按 movie_id 数值升序输出，并剔除已评分与已收藏电影。

该方法的优点是物品侧信息即可工作、结果可解释（能说明“因为你喜欢动画与家庭题材”）、对交互稀疏的用户仍可用；缺点是特征表达能力受限于标签体系，容易出现类型对但具体片子不合适的结果，且存在“越推越窄”的同质化风险。若用户没有任何正反馈，画像为空，本实现如实返回空列表而不使用随机结果填充。

表 2-1 2.2 基于内容的推荐

项目	记录 1	记录 2	记录 3	记录 4
特征表示	类型标签	TF-IDF 权重	L2 归一化	余弦相似度
画像构造	仅 rating ≥ 4.0	权重 max(rating−3, 0.1)	加权平均	无正反馈则返回空
关键代码	recommender/student/content.py	弧度归一化与 idf 计算	画像聚合	排序与过滤
已验证事实	ml:14 首页 10 条	与已评分零交集	同分按 ID 升序	冷启动分区不返回随机结果

2.2.2 证据与分析

内容推荐在冻结协议下的验证集 Precision@10 为 0.0041、Recall@10 为 0.0075，是五类算法中最低的一档。这个结果与特征容量一致：真实类型标签只有 19 个取值，一部电影通常只带 2—3 个标签，不同电影的类型向量高度相似，因此按余弦排序得到的候选之间区分度有限。与之对照，ItemCF 在同一协议下达到 0.0553，说明在本数据集上共现行为比类型标签携带了更多区分信息。

该结果并不意味着内容推荐没有价值。在冷启动场景（第 4.6 节）与“用户只有少量行为”的场景中，类型画像仍是最快可用的个性化依据，且它的可解释性最好。第 4.6 节的冷启动实现正是把“有正反馈时用类型画像、没有正反馈时用全站正反馈热门榜”两段策略组合起来，从而在两类用户上都给出明确、可解释的结果。

2.3 协同过滤与矩阵分解

2.3.1 方法与事实说明

协同过滤不依赖物品属性，而是从行为共现中学习。基于用户的协同过滤（UserCF）寻找相似用户，用相似用户的行为预测目标用户；基于物品的协同过滤（ItemCF）则计算物品之间的相似关系，把用户历史物品的邻居按相似度加权求和。两者都用余弦

或皮尔逊相关系数度量相似度，区别在于相似度矩阵的规模与稳定性：用户数量变化快，物品相对稳定，因此工业界常用 ItemCF。

本项目的 ItemCF 把每个用户的正反馈电影（rating ≥ 4.0）视为一次共现证据，物品间相似度用余弦公式计算，并设置三道阈值抑制噪声：物品至少被 20 位用户正反馈（min_item_support = 20），两物品至少共同出现在 5 位用户的正反馈中（min_cooccurrence = 5），每部电影最多保留 50 个邻居（max_neighbors = 50）。推荐时对用户历史电影的邻居按相似度加权求和，按分数降序输出并剔除已交互电影；没有任何历史正反馈时返回空列表。

矩阵分解把评分矩阵近似为两个低秩矩阵的乘积，用户 u 对物品 i 的预测为 p_u · q_i。相对邻域方法，它的优势是能捕捉隐语义、参数量可控、可用交替最小二乘（ALS）或随机梯度下降求解；代价是训练成本更高、对冷启动用户需要额外处理。本项目使用 Spark MLlib 的 ALS 以显式评分模式训练（implicitPrefs = false），rank = 32、regParam = 0.05、maxIter = 15、seed = 20261022；对训练集中没有隐向量的用户，用其历史电影的因子按评分权重做 fold-in 回填，从而在不额外训练的情况下给出个性化结果。

多路召回还需要统一排序，常见做法有线性加权、学习排序（LambdaMART 等）与名次融合。本项目选择 Reciprocal Rank Fusion（RRF）：对第 s 路召回的第 rank 名条目计分 1/(k + rank)（k = 60），乘以该路权重后求和。RRF 只使用名次、不使用原始分数，因此天然兼容不同量纲的算法，权重可以用离线实验确定，且计算代价极低。

表 2-2 2.3 协同过滤与矩阵分解

项目	记录 1	记录 2	记录 3	记录 4
方法	UserCF	ItemCF	矩阵分解（ALS）	RRF 融合
核心思想	相似用户的行为	共现行为上的物品相似度	评分矩阵低秩分解	多路名次加权求和
本项目采用	未采用（物品更稳定）	余弦 + 三重阈值	rank 32、reg 0.05、迭代 15	k = 60、权重由实验确定
局限	用户规模变化快	稀疏物品无邻居	冷启动用户需回填	依赖各路召回质量

2.3.2 证据与分析

在同一冻结协议下，ItemCF 的验证集 Precision@10 为 0.0553、Recall@10 为 0.0978，是五类算法中最高的；ALS 为 0.0414 / 0.0672，位列第二；两者加权融合（ItemCF 权重 6、ALS 权重 1）得到 0.0558 / 0.0974，Precision 略高于单路 ItemCF，Recall 略低。这组数字说明：在本数据集的物品规模与交互密度下，邻域方法比隐向

量方法更契合 Top-N 命中率指标。

文献中常见的结论（矩阵分解通常优于邻域方法）在本数据集上并未成立，原因有两点：其一，本项目评价 Top-10 命中率而非评分误差，隐向量在评分拟合上的优势不一定转化为名次优势；其二，物品侧共现信号集中（热门电影邻居丰富），ItemCF 的加权求和与命中率目标更直接对齐。这一差异也说明“方法选择必须以本数据集上的证据为准”。

2.4 系统开发关键技术

2.4.1 方法与事实说明

后端使用 Python 3.12 与 FastAPI 0.143：Pydantic 2.14 负责请求与响应校验（严格模式、禁止未声明字段），SQLAlchemy 2.0.54 负责数据库访问，PyMySQL 作为驱动，Uvicorn 作为 ASGI 服务器。接口统一挂在 /api/v1 前缀下，错误响应统一为 {error: {code, message, details}, request_id} 结构，写操作要求携带 CSRF 令牌与同源 Origin，登录态使用 HttpOnly Cookie，限流与审计日志由课程既有中间件提供。

前端使用 Vue 3 + TypeScript + Vite，界面组件库为 Element Plus，状态与请求封装在 src/api.ts 与 composable 中，页面包含首页推荐、电影库、详情页、登录注册、个人中心、推荐实验室与冷启动引导页。样式为项目自维护的浅色主题（CSS 变量集中定义在 theme.css，页面与组件样式追加在 style.css 末尾），静态资源由 Nginx 提供，接口请求经同源代理访问后端。

数据存储使用 MySQL 8.0.46，字符集 utf8mb4_0900_bin，库名形如 rec_course_<manifest 前 12 位>_{business|evaluation}，业务库与评价库严格分离：业务库容纳真实用户行为与推荐结果，评价库容纳冻结的 train/validation/test 与冷启动分区。运行时使用 Windows 11 + WSL2 + Docker Desktop，课程离线镜像（cinelab-offline/api:1.3.0-a1、web:1.3.0-a1、mysql:8.0.46-a1）作为底座，compose 按“基础文件 + lab5 覆盖 + lab6 覆盖”分层启动；后端容器只读根文件系统、禁用权限提升，数据包与模型目录只读挂载。

扩展模块的关键技术是 Apache Kafka 3.9.0（KRaft 单节点，容器内 kafka:9092、宿主 127.0.0.1:29092）与 kafka-python；离线训练使用 Spark 3.5.3 本地模式（宿主 JDK 11）与 numpy/pandas/scipy；自然语言助手通过 httpx 调用兼容 OpenAI 协议的 DeepSeek 接口，模型不直接访问数据库，事实全部来自四个只读工具。海报采用按需抓取与本地缓存：后端以电影的 TMDB 标识访问公开影片页，抓取海报图片字节

并缓存，容器内落盘到挂载的 posters 目录，未命中时回退到确定性生成的矢量海报。

- 后端：FastAPI + Pydantic + SQLAlchemy + MySQL 8；容器内不含 numpy/pandas，算法推理只用标准库。
- 前端：Vue 3 + TypeScript + Vite + Element Plus；浅色主题与紧凑排布由项目维护。
- 部署：Windows 11 + WSL2 + Docker Desktop，课程离线镜像 + compose 覆盖文件分层启动，只读挂载。
- 扩展：Kafka 3.9.0、kafka-python、DeepSeek 兼容接口、Spark 3.5.3 本地训练、海报缓存。

2.4.2 证据与分析

技术栈的可用性在项目中逐项验证过：api 容器内能导入 httpx、sqlalchemy 与 fastapi（无 requests、无 OpenAI SDK），因此助手实现选择 httpx；容器出网测试返回 api.deepseek.com 的 HTTP 401（网络可达、缺密钥），配置密钥文件后真实调用返回 200；Kafka 容器以 KRaft 单节点启动并通过 kafka-topics.sh 健康检查；Spark 仅在宿主训练阶段使用，训练产物（ALS 因子 JSON，约 4 MB）提交到仓库后在容器中直接读取推理。

限制有两条需要记录：课程离线镜像不包含 numpy、pandas、scipy 与 Java，凡需要这些依赖的环节都必须在宿主完成并把结果作为工件提交；本机 Docker Hub 不可达，课程文档中的联网重建路径不可用，扩展用镜像改为离线安装包与本地派生镜像完成。这两条限制在第 7.2 节展望中作为后续改进项列出。

3 系统需求分析与总体设计

3.1 系统需求分析

3.1.1 方法与事实说明

系统角色有四类。游客可以浏览首页、电影库与详情页，但不能写入行为；注册用户可以评分、收藏、查看个性化推荐与推荐实验室；演示者可以使用课程提供的只读历史用户（ml:14、ml:55 等）复现冻结行为下的推荐结果；评审者可以在本机复现全部命令与证据。四类角色的权限边界由课程既有的认证与 CSRF 机制保证，本项目只在其上增加推荐相关能力。

功能需求按“浏览—行为—推荐—验证”四条线组织：F1 电影目录与检索（类型、年份、关键词、分页）、F2 电影详情、F3 评分（0.5—5.0，步长 0.5）、F4 收藏与

取消、F5 五类算法的个性化推荐、F6 历史用户只读演示、F7 推荐实验室（算法切换、用户切换、历史与结果对照）、F8 离线评价结果展示、F9 融合与短期兴趣扩展、F10 自然语言助手。每项需求都对应到具体接口与页面，并写入集成检查脚本进行回归断言。

非功能需求包括五条：可复现（数据版本与模型摘要固定，命令可重跑）、可解释（每条结果带算法、模型版本、分数与召回来源）、安全（写操作需 CSRF 与同源校验，容器只读根文件系统、禁止权限提升、密钥以只读文件挂载）、可移植（compose 分层覆盖，Windows/WSL2 环境一键启动）、离线优先（课程镜像离线导入，扩展依赖本地安装包）。性能方面，本地环境的推荐接口响应在 0.1 s 量级，集成检查记录的整轮 31 项断言约数十秒完成。

- 输入：用户会话（账号或只读历史用户）、电影目录与类型、正反馈定义（rating ≥ 4.0）。
- 处理：五路召回 + 名次融合 + 短期兴趣重排，全部剔除已交互电影后输出 Top-10。
- 输出：带算法、模型版本、分数、召回来源与名次的推荐列表；离线评价表；行为与助手状态。
- 约束：冻结数据不可改；电影元数据缺失（导演、主演、时长、简介、海报为空）；容器无 numpy/pandas。

3.1.2 证据与分析

需求与实现的对照关系可以直接核对：F1—F4 由课程既有接口与页面承载，本项目只做展示与排布优化；F5、F6、F7 由 recommender/student 下的五个 Provider 与推荐实验室页面实现；F8 由离线评价脚本与前端评价表实现；F9 由 fusion.py、interest.py、behavior.py 与对应前端面板实现；F10 由 llm.py、assistant.py 与助手面板实现。

取舍说明：需求中没有“公网部署”与“联网重建课程离线镜像”两项，它们在报告中明确标注为未做；需求中也没有“补全电影元数据”，因为补造数据会破坏“证据必须来自真实数据”的原则。这三处取舍在 7.2 节作为不足与展望列出。

3.2 系统总体架构设计

3.2.1 方法与事实说明

系统采用分层架构。表现层是 Nginx 托管的 Vue 3 单页应用，负责页面渲染、表单校验与请求封装；服务层是 FastAPI 应用，按“路由层—领域模块—契约层”组织：

路由层提供 /api/v1 下的 REST 接口，领域模块（fusion、interest、behavior、assistant、posters）承载扩展能力，契约层是 recommender 包，包含数据结构定义（contracts）、模型清单校验（validation）、冻结数据读取（data）、离线评价协议（evaluation）与业务历史读取（business）。数据层是 MySQL 业务库与评价库，以及 models 目录下的模型工件与 posters 目录下的海报缓存。

算法调用链固定为：路由层构造 RecommendationRequest → 契约层 recommend() 查找已注册 Provider → 校验请求与模型清单 → 调用 Provider.recommend() → 校验结果 DTO（数量、去重、名次连续、同分规则、身份一致性）→ 返回前端。这条链上的任一环节失败都会返回结构化错误（如 ALGORITHM_NOT_IMPLEMENTED、MODEL_BINDING_INVALID、DATA_IDENTITY_INVALID），而不是降级成看似正常的结果。

扩展层与主链路解耦：行为事件通过 Kafka 生产/消费，消费端把短期兴趣写入 JSON 状态文件，后端读取文件后用于重排；自然语言助手在模型与数据库之间插入白名单工具层，模型只能看到工具返回的事实；海报缓存由后端按需抓取并落盘，未命中时回退到矢量海报。三条扩展都不修改课程冻结的业务表结构与既有接口契约。

部署视图由三个 compose 文件叠加：compose.yaml 提供 web、api、db 三个基础服务；deployment/lab5/compose.lab5.yaml 增加 Kafka 容器并为 api 注入事件与兴趣状态环境变量；deployment/lab6/compose.lab6.yaml 追加模型密钥挂载、自适应权重参数与海报目录挂载，并用本地派生镜像替换 api 与 web。数据包与模型目录以只读方式挂载进容器，密钥目录以只读方式挂载，运行日志与状态文件写入宿主目录。

3.2.2 证据与分析

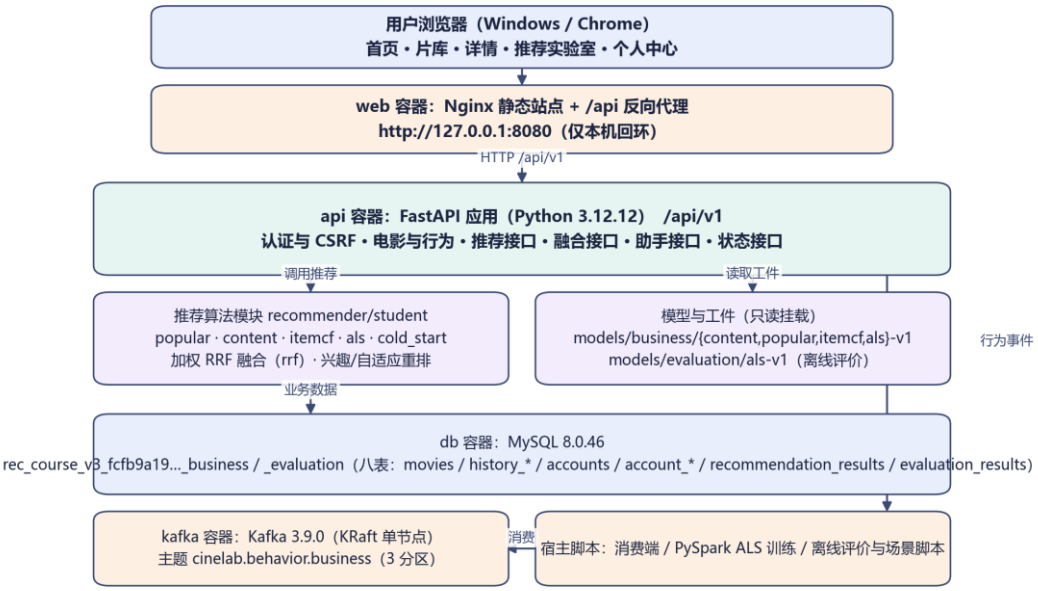


图 3-1 系统总体架构（表现层—服务层—契约层—数据层与扩展层）

架构中的三个设计决策值得说明理由。第一，算法实现放在 recommender 包而非 HTTP 路由内，目的是让离线评价与在线服务共用同一份实现，避免“线上一个版本、离线一个版本”导致指标与系统不一致；第二，业务库与评价库分离、models/business 与 models/evaluation 分离，目的是防止把在测试集上拟合过的模型用于在线推荐，这一约束由契约层的清单校验强制执行；第三，扩展能力一律不改课程既有表结构，短期兴趣与海报缓存写文件、写 Kafka 主题，代价是状态不在数据库事务内，收益是课程冻结的数据契约保持完好。

运行证据：四个容器（web、api、db、kafka）在集成检查期间全部处于 healthy 状态；/health/ready 返回 status=ready 及数据版本 rec-course-v3-train5-1.0.0、split temporal-v1-cold500-train5-v3、schema student-eight-v3；api 容器只读根文件系统与 no-new-privileges 生效，容器内无法定位写操作。

3.3 系统功能模块设计

3.3.1 方法与事实说明

功能模块划分为八个：认证与会话（课程既有）、电影目录与详情（课程既有，前端重排）、用户行为（评分与收藏，课程既有，新增事件发布）、推荐服务（五个 Provider 与融合）、离线评价（脚本、工件与前端评价表）、实时行为与短期兴趣（Kafka 生产、消费端、状态文件、重排接口）、自然语言助手（受控工具调用）、展示与海

报（海报缓存、主题与布局）。每个模块都有对应的后端文件、前端组件与验证脚本。

前端页面与组件映射为：HomeView（首页推荐与冷启动入口，使用 AlgorithmPanel）、MoviesView（电影库与分页，使用 MovieCard 与 MoviePoster）、DetailView（详情与行为表单）、AuthView（登录注册）、ProfileView（我的评分与收藏）、ExperimentsView（推荐实验室，组合 AlgorithmPanel、FusionPanel、AssistantPanel 与评价表）、ColdStartView（冷启动引导）。组件之间通过 src/api.ts 统一请求，类型定义集中在 src/types.ts。

后端路由模块映射为：auth.py（会话）、catalog.py（目录与历史用户）、activity.py（评分收藏与事件发布）、fusion.py（融合与自适应权重）、behavior_api.py（行为状态与兴趣）、assistant.py（助手）、experiments.py（算法注册表与评价入口）、posters.py（海报与缓存状态）。路由注册顺序被固定下来，fusion 必须先于 experiments 注册，否则 /me/recommendations/{algorithm} 会抢占 “fusion” 路径。

表 3-1 3.3 系统功能模块设计

项目	记录 1	记录 2	记录 3	记录 4
模块	推荐服务	离线评价	实时行为	自然语言助手
后端文件	recommender/student/*.py、backend/app/fusion.py	recommender/evaluation.py、scripts/run_lab3_evaluation.py	backend/app/behavior.py、behavior_api.py、interest.py	backend/app/llm.py、assistant.py
前端组件	AlgorithmPanel、FusionPanel	ExperimentsView 评价表	FusionPanel 兴趣区块、行为面板	AssistantPanel
验证方式	契约校验 + 集成检查	官方 check 命令 + 指标复算	场景脚本 + 状态对照	助手四场景脚本

3.3.2 证据与分析

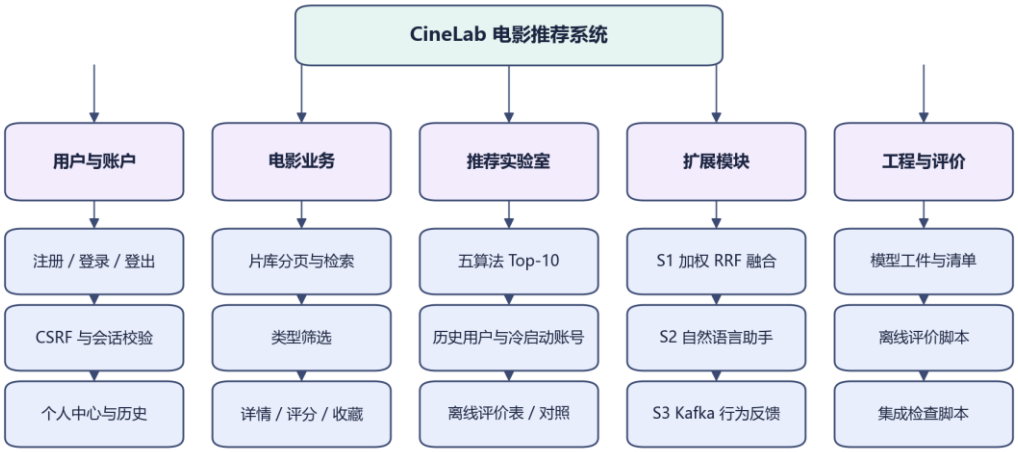


图 3-2 系统功能模块与数据流（八个模块及其依赖）

模块划分的效果在集成检查中得到验证：31 项断言覆盖了模块之间的衔接点，包括“算法注册表五项齐备”“推荐结果与已交互集合零交集”“融合结果带召回来源”“兴趣状态可被读取且版本递增”“助手返回的工具结果全部来自真实数据”“评价表的行数与算法数一致”。任一模块接口被破坏，对应断言即失败并给出具体差异，这也是模块划分（而非单一巨型脚本）带来的可测试性收益。

3.4 数据库设计

3.4.1 方法与事实说明

业务库与评价库都采用课程字段冻结的八张表结构。业务库包含 movies（电影目录，主键 movie_id，字段含标题、上映年份、类型、TMDB/IMDb 标识）、history_users（只读历史用户）、history_ratings（历史评分）、accounts（注册账号）、account_ratings（账号评分）、account_favorites（账号收藏）、recommendation_results（推荐结果批次，含由 SHA-256 生成的 batch_id 生成列）、evaluation_results（离线评价结果）。评价库包含相同的目录表结构与冻结分区数据（train/validation/test 与冷启动分区），供离线评价脚本读取。

关键约束有五条：主键与唯一键保证幂等导入；评分字段为 DECIMAL(2,1) 并限制 0.5—5.0；账号与历史用户通过 subject_kind（app/ml）隔离；recommendation_results 的 batch_id 由 runtime_mode、dataset_version、split_version、algorithm、model_version、batch_token 与 subject_kind 拼接后取 SHA-256 生成，保

证同一批次可追溯；导入顺序固定为 movies → history_users → history_ratings → accounts → account_ratings → account_favorites → recommendation_results → evaluation_results，避免外键与唯一键冲突。

本项目的扩展不新增业务表。短期兴趣状态写入 JSON 文件（interest.json 与事件流水 events.jsonl），海报缓存写入 posters 目录，融合结果只在响应中返回而不落库。这样做的代价是缺少数据库层面的历史查询能力，收益是课程的数据契约完全不被改动，评审可以仅通过接口与文件证据复核扩展功能。

3.4.2 证据与分析

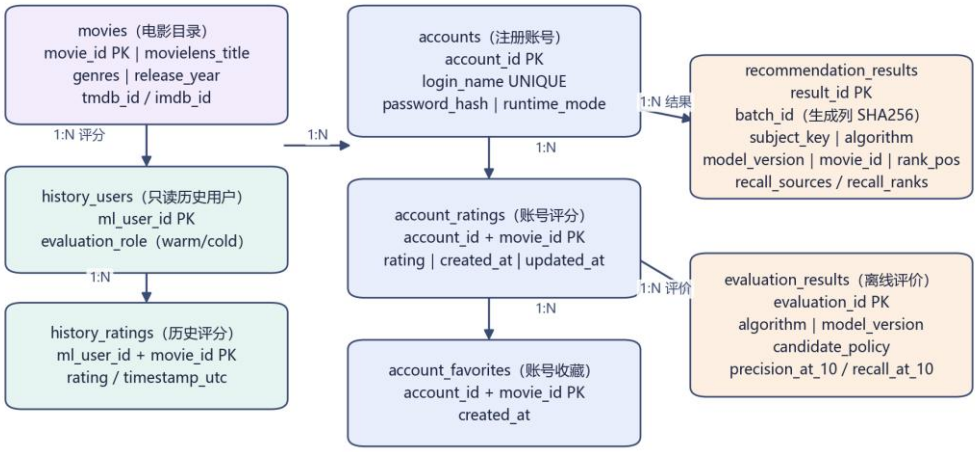


图 3-3 数据库主要表与关系（业务库与评价库同构，数据隔离）

真实行数用于核对数据完整性：movies 8,459 部；history_users 5,000 位（其中 4,500 位有历史行为、500 位为冷启动用户）；history_ratings 798,775 条；accounts 40 个（本项目测试期间创建）；account_ratings 51 条；account_favorites 11 条。评价库分区行数为 train 583,502、validation 70,366、test 67,792、cold-history 77,115，冷启动四个分区各 499 位有效用户。这些数字与课程数据包 README 的声明一致，并在集成检查中作为断言条件。

3.5 推荐服务接口设计

3.5.1 方法与事实说明

推荐服务接口以 /api/v1 为前缀，ID 一律使用十进制字符串。核心 DTO 有三个：RecommendationRequest（subject_kind、subject_id、limit、runtime_mode、

dataset_version、split_version）、RecommendationItem（movie_id、rank、algorithm、model_version、score、recall_sources、recall_ranks、interest 相关字段）、RecommendationResult（status、algorithm、model_version、版本三元组、runtime_mode、batch_id、generated_at、items）。契约层对 DTO 做严格校验：字段不得多余、limit 必须满足、rank 连续、movie_id 必须在目录内、同分必须按数值 ID 升序。

接口分为五组。目录与行为组：GET /genres、GET /movies、GET /movies/{id}、PUT/GET/DELETE /me/ratings 与 /me/favorites。推荐组：GET /me/recommendations/{algorithm}（algorithm ∈ popular/content/itemcf/als/cold_start）与 GET /demo/history-users/{ml_user_id}/recommendations/{algorithm}。融合组：GET /experiments/fusion/sources、GET /me/recommendations/fusion（参数 algorithms、weights、limit、interest、adaptive）与历史用户同名端点。扩展组：GET /me/behavior/interest、GET /experiments/behavior/status、GET /me/assistant/query、GET /experiments/assistant/status、GET /movies/{id}/poster、GET /experiments/poster-cache。评价组：GET /experiments/algorithms（注册表）与 GET /experiments/evaluations（按契约返回 501 EVALUATION_NOT_IMPLEMENTED，指标由前端读取构建期数据展示）。

错误与安全约定：错误响应统一为 {error:{code,message,details},request_id}；常见错误码包括 VALIDATION_ERROR（422）、UNAUTHORIZED（401）、FORBIDDEN（403）、MOVIE_NOT_FOUND（404）、HISTORY_USER_NOT_FOUND（404）、ALGORITHM_NOT_IMPLEMENTED（501）、MODEL_BINDING_INVALID / DATA_IDENTITY_INVALID（500 级）、ASSISTANT_UNAVAILABLE（503）；写操作要求 Origin 同源且携带 x-csrf-token，登录态为 HttpOnly Cookie；GET 接口不接受写操作语义，POST 接口不接受查询参数。

表 3-2 3.5 推荐服务接口设计

项目	记录 1	记录 2	记录 3	记录 4
推荐端点	/me/recommendations/{algorithm}	/demo/history-users/{id}/recommendations/{algorithm}	/me/recommendations/fusion	/experiments/fusion/sources
关键参	limit 1—50	algorithm 五值枚举	algorithms/weights/limit/interest/adaptive	—

数				
返回要点	items + 模型版本 + 来源	同上，只读历史用户	pool_size 、filtered_seen、sources、interest、adaptive	默认来源与默认权重
验证状态	集成检查断言 10 条且零交集	ml:14、ml:55 双用户断言	权重与名次对照实验	前端与脚本均读取

3.5.2 证据与分析

接口实测证据包括：GET /experiments/algorithms 返回五个算法全部 registered；GET /me/recommendations/content 返回 10 条且与已评分零交集；GET /me/recommendations/fusion?algorithms=itemcf,als&weights=6,1&interest=1&adaptive=1 返回 pool_size、filtered_seen、sources（含各路基权重与自适应权重）与 interest 区块；GET /experiments/evaluations 按契约返回 501 与说明，前端据此显示“评价指标由构建期数据展示”的提示，而不是伪造在线指标。

接口设计的一个取舍需要说明：/experiments/evaluations 在课程契约中就不提供在线计算，因此本项目没有把它改成在线接口，而是把离线评价结果发布为构建期 JSON（frontend/src/data/lab3-metrics.json 与 lab4-fusion.json）供页面展示，同时保留后端契约行为。这样既满足“页面能看到指标”，又不违反课程接口约定。

4 推荐算法设计与实现

4.1 数据集与预处理

4.1.1 方法与事实说明

数据集来自课程冻结数据包 rec-course-v3-train5-1.0.0（源自 MovieLens 32M 的电影与评分，按课程规则重新划分）。目录包含 8,459 部电影（要求训练支持度不低于 5），有效历史用户 5,000 位，其中 4,500 位有历史行为、500 位为冷启动用户。业务演示用评分文件 history_ratings.csv 共 798,775 行；离线评价使用的训练集 train.csv 共 583,502 行、验证集 validation.csv 70,366 行、测试集 test.csv 67,792 行、冷启动历史 cold-history.csv 77,115 行，冷启动四个分区各含 499 位有效用户（validation 有效用户 4,209 位、test 4,198 位）。

正反馈定义统一为 rating ≥ 4.0。候选集合按分区生成：验证集候选 = 目录 − 该用户在训练集中的评分；测试集候选 = 目录 − 训练 − 验证；冷启动 k 分区候选 =

目录 − 该用户的 support 文件。这一规则保证三个分区之间没有信息泄漏，也保证“已看过但没评分”的电影不会被当作未交互物品。数据包中允许离线读取的文件被白名单限定（training、movies.csv、training-catalog.csv 与各分区文件），业务全量评分与 cold-history.csv 在评价模式下会被读取器拒绝。

预处理分三步。第一，校验身份：读取数据包 manifest.json 并比对 SHA-256，确认 dataset_version、split_version、schema_version 与课程契约一致；第二，构造特征：从 movies.csv 取电影标题、年份与类型标签，过滤“(no genres listed)“占位标签，得到 19 个真实类型取值，并按类型出现频率计算 IDF 权重；第三，构造训练视图：把 training 文件按 userId 分组得到每位用户的正反馈物品集合，供 ItemCF 共现统计与 ALS 训练使用。特征文件只允许使用 movies.csv 与 training-catalog.csv 两个来源，模型清单中的 feature_files 字段会校验这一点。

表 4-1 4.1 数据集与预处理

记录项	输入与数据范围
数据身份	dataset_version=rec-course-v3-train5-1.0.0 ； split_version=temporal-v1-cold500-train5-v3 ； schema_version=student-eight-v3
目录与用户	movies.csv 8,459 部；history_users 5,000 位（4,500 暖 + 500 冷启动）
分区规模	train 583,502；validation 70,366（有效用户 4,209）；test 67,792（4,198）；cold-history 77,115
正反馈与候选	rating ≥ 4.0；候选 = 目录 − 该分区累计已评分物品
实现文件与提交	recommender/data.py 、recommender/student/evaluation_data.py（第 3 次提交）
运行与验证证据	集成检查断言数据身份；check evaluation-inputs 通过；评价表行数与分区一致

4.1.2 证据与分析

数据侧的三个事实直接影响算法设计。第一，交互密度不均：4,500 位暖用户贡献了 583,502 条训练评分，平均每位用户约 130 条，足以支撑物品共现统计；第二，类型标签稀疏：19 个类型取值、每部电影 2—3 个标签，决定了内容推荐只能作为弱特征；第三，冷启动分区用户完全没有可用的历史行为，因此必须有一个不依赖个人信息化的兜底路径。

本节的验证手段是把数据身份写入运行证据：/health/ready 返回的 dataset_version 与 split_version 与数据包一致；评价脚本生成的 task_sha256 与

user_ids_sha256 可用于核对同一分区两次运行是否面对完全相同的数据；模型清单中的 training_files 摘要与数据包中的实际文件一致，否则 check_model 失败。

4.2 Popular 热门推荐

4.2.1 方法与事实说明

Popular 是无个性化的热门榜，用于回答“大多数人现在喜欢什么”。它的输入是训练文件中的正反馈记录，按电影统计两个量：positive_support（正反馈人数）与 mean_rating（平均分）。发布到前端的 score 采用归一化正反馈人数，即 score = positive_support / top_positive_support，使榜首为 1.0，其余按比例落在 (0, 1]；平均分作为工件的诊断列保留，用于人工核对。排序规则为 score 降序，同分时按 movie_id 数值升序，符合课程契约对并列的要求。

过滤与回退规则有两条：正反馈人数少于 20 的电影不进入榜单（min_positive_support = 20），以排除长尾偶然命中；推荐时剔除当前用户已评分与已收藏的电影，若剔除后不足 limit 则按榜单顺序继续补充。工件 popular.json 保留全部 8,459 部电影的统计值（约 830 KB），推理时只读该文件，不访问 Spark 或数据库。

表 4-2 4.2 Popular 热门推荐

记录项	输入与数据范围	核心公式与符号定义	关键参数与取值	过滤与回退规则	实现文件与提交	运行与验证证据
Popular	训练文件正反馈（rating ≥ 4.0）；全目录 8,459 部	score = positive_support / top_positive_support；排序 score↓，同分 movie_id↑	min_positive_support = 20；max_recommendations = 10	剔除已评分与收藏；榜单不足则继	recommender/student/popular.py；models/business/popular-v1/（第 2 次提交）	权威榜单前三：318（2234 人，1.0）、296（1946 人，0.871）、356（1839 人，0.823）

				续下探；无工件返回空结果		

4.2.2 证据与分析

Popular 在验证集上的 Precision@10 为 0.0342、Recall@10 为 0.0578，测试集为 0.0312 / 0.0516。它高于内容推荐的 0.0041，说明“全站热度”本身携带了真实信号：被大量用户打高分的电影更可能出现在任何人的相关集合里。在冷启动 k = 0 分区（用户完全没有可用历史）中，Popular 的 Precision@10 达到 0.3369，是所有算法在冷分区上的最高值——这也解释了为什么冷启动策略把热门榜作为无历史时的兜底。

限制同样明确：Popular 对所有人返回相同列表（只做已看过滤），无法体现个人偏好；在交互稀疏的冷门电影上永远没有机会。因此它在本项目中只作为冷启动兜底与对照基线，不承担个性化推荐职责。

4.3 Content 基于内容的推荐

4.3.1 方法与事实说明

Content 使用类型标签的 TF-IDF 表示。第一步构造物品向量 v_i：对电影 i 的每个类型标签 g 取 idf(g) = ln(总电影数 / 含该标签的电影数) + 1，得到稀疏向量并做 L2 归一化，使余弦相似度等价于内积。第二步构造用户画像 p_u：取该用户历史正反馈电影（rating ≥ 4.0）的向量加权平均，权重 w = max(rating − 3.0, 0.1)，即 4.0 分权重 1.0、5.0 分权重 2.0，3.0 分及以下按 0.1 处理。第三步排序：计算 cos(p_u, v_i) = p_u · v_i，按分数降序、同分按 movie_id 数值升序输出。

过滤与回退规则：剔除已评分与已收藏电影；用户没有任何正反馈时画像为空，直接返回空列表（0 条）而不使用热门榜或随机结果顶替，由冷启动算法承担兜底。工件 movie-genres.json 保存类型词表、IDF 表与每部电影的类型向量，推理阶段只用标准库完成向量点积，不依赖 numpy。评价模式下的同一算法使用 train.csv 训练，

模型版本区分为 content-genre-tfidf-eval-v1，业务与评价两套工件目录分离。

表 4-3 4.3 Content 基于内容的推荐

记录项	输入与数据范围	核心公式与符号定义	关键参数与取值	过滤与回退规则	实现文件与提交	运行与验证证据
Content	movies.csv 的类型标签（19 个真实取值）；用户正反馈历史	v_i 由 IDF 加权并 L2 归一化；p_u = Σ w_j v_j / Σ w_j；score = cos(p_u, v_i)	w = max(rating − 3.0, 0.1)；相似度 cosine；max_recommendations = 10	剔除已评分与收藏；无正反馈返回空；占位标签 (no genres listed) 被过滤	recommender/student/content.py；models/business/content-v1/movie-genres.json（第 1 次提交）	ml:14 首页 10 条且与已评分零交集；冷启动分区 k = 0 返回 0 条（如实空结果）

4.3.2 证据与分析

Content 在验证集上的 Precision@10 为 0.0041、Recall@10 为 0.0075，测试集 0.0037 / 0.0070，是五类算法中最低的一档。可核对的原因有两个：类型标签只有 19 个

取值，不同电影的类型向量高度相似，导致排序区分度低；课程协议的正反馈物品集合中，同类型但不同系列的电影数量多，纯类型匹配容易把“类型对但口碑一般”的电影排在前面。

它仍然承担两项职责：在冷启动有少量行为时提供可解释的个性化（“因为你喜欢动画与家庭题材”），以及在扩展模块中作为融合的一路候选（第 4.7 节的权重实验显示，加入 content 会拉低融合精度，因此最终默认来源只保留 ItemCF 与 ALS）。

4.4 ItemCF 协同过滤

4.4.1 方法与事实说明

ItemCF 从行为共现构造物品相似度。实现分两步：统计阶段，把训练文件中每位用户的正反馈电影视为一次共现（每位用户的集合内部两两计数），得到物品对共现次数；发布阶段，对满足 min_item_support = 20（物品至少被 20 位用户正反馈）与 min_cooccurrence = 5（两物品至少共同出现在 5 位用户的正反馈中）的物品对计算余弦相似度，并按相似度降序为每部电影保留最多 50 个邻居（max_neighbors = 50），写入 item-similarities.json。

推理阶段：取当前用户历史正反馈电影集合 S，对 S 中每部电影的邻居累加相似度（同一候选被多个历史物品命中则相似度相加），得到候选分数，按分数降序、同分按 movie_id 数值升序输出，并剔除已评分与已收藏电影。工件覆盖 3,000 部有足够支持的物品（约 3.24 MB），没有邻居的用户返回空结果。为支持融合模块获取更深候选，ItemCF 与其余 Provider 的返回上限统一放宽到 50 条（契约默认仍为 10 条），这一点在实现注释中记录了理由。

表 4-4 4.4 ItemCF 协同过滤

记录项	输入与数据范围	核心公式与符号定义	关键参数与取值	过滤与回退规则	实现文件与提交	运行与验证证据
ItemCF	训练文件二值化正反馈（rati	sim(i, j) = |U_i ∩ U_j| / sqrt(|U_i| · |U_j|)；score(x) = Σ_{i∈S} sim(i, x)	min_item_support = 20；min_cooccurrence = 5；max_neighbors = 50；返回	剔除已评分与	recommender/student/itemcf.py ；models/business/itemcf-v1/item-similarities.json（第 2 次提交）	3,000 部物品带邻居；验证集 Precision@10

	ng ≥ 4.0）；共现统计		上限 50	收藏；无邻居返回空；支持度不足的物品不进入工件		0.0553 为五类最高；融合候选池 57 条（itemcf 30 + als 30，去重后）

4.4.2 证据与分析

ItemCF 在验证集上的 Precision@10 为 0.0553、Recall@10 为 0.0978，测试集 0.0517 / 0.0902，是五类算法中表现最好的一档；在冷启动 $k \geq 1$ 的分区上也保持领先（$k = 1$ 时 0.1858）。原因可以从数据侧解释：训练集每位暖用户平均约 130 条评分，物品共现信号充足，且 Top-10 命中率指标与“按共现加权排序”的目标高度一致，模型不需要拟合精确分数。

限制有三点：物品支持度不足的长尾电影没有邻居，无法被推荐；用户历史正反馈很少时分数不稳定；相似度矩阵随数据更新需要重建。工程上把相似度矩阵发布为 JSON 工件，代价是 3.24 MB 的仓库增量，收益是推理只需标准库且结果可复现（文件摘要写入模型清单）。

4.5 ALS 矩阵分解

4.5.1 方法与事实说明

ALS 使用 Spark MLlib 的显式评分模式（implicitPrefs = false）在训练集上做矩阵分解，学习用户隐向量 p_u 与物品隐向量 q_i，预测分数为二者内积。训练参数为

rank = 32、regParam = 0.05、maxIter = 15、seed = 20261022，coldStartStrategy = drop。训练完成后导出物品因子、用户因子与物品支持度到 als-factors.json（业务版约 4.21 MB、评价版约 4.14 MB），推理阶段只读 JSON，不再需要 Java 或 Spark。

两个工程细节决定可用性。第一是用户回填：对训练集中没有隐向量的用户（例如新注册账号），用其历史正反馈电影的物品因子做评分加权平均作为该用户的近似向量（fold-in），从而在不重训的前提下给出个性化结果。第二是候选支持度下限：训练文件内部统计出的 item_support 随工件一起发布，推理时把支持度低于 500 的电影排除出候选（candidate_support_floor = 500）。这一措施针对的是隐向量对冷门物品估计不足的问题：支持度低的物品因子方差大，容易在 Top-10 里形成噪声。

过滤与回退：剔除已评分与已收藏电影；用户既无隐向量也无法回填（没有任何正反馈）时返回空结果；支持度未知的电影按下限规则处理（视为不足）。模型清单中记录了训练网格、支持度下限网格与最终验证集指标，作为参数选择的证据。

表 4-5 4.5 ALS 矩阵分解

记录项	输入与数据范围	核心公式与符号定义	关键参数与取值	过滤与回退规则	实现文件与提交	运行与验证证据
ALS	训练文件显式评分（1.0—5.0）；物品支持度统计	score = p_u · q_i；无因子用户 p_u = Σ w_j q_j / Σ w_j（fold-in）	rank 32；regParam 0.05；maxIter 15；seed 20261022；support floor 500	剔除已评分与收藏；支持度 < 500 的候选被	recommender/student/als.py；models/{business,evaluation}/als-v1/als-factors.json（第 3 次提交）	验证集 Precision@10 0.0414；支持度下限对照 0.0054→0.0416；fold-in 在注册账号上返回 10 条

				排除；无正反馈返回空		

4.5.2 证据与分析

ALS 在验证集上的 Precision@10 为 0.0414、Recall@10 为 0.0672，测试集 0.0396 / 0.0598，位列第二。参数实验显示两点规律：隐向量维度是主要影响因子，rank 从 8 提升到 32 使 Precision@10 从 0.0003 提高到 0.0054（在同样使用全目录候选时），说明低维隐向量不足以表达本数据集的偏好差异；支持度下限把候选从 8,459 部收缩到 193 部后，Precision@10 从 0.0054 提升到 0.0416，说明“先过滤不可靠物品、再排序”比“全目录排序”更符合 Top-10 命中目标。

该结果也解释了 ALS 为何低于 ItemCF：ItemCF 的相似度由共现直接支撑，而 ALS 的隐向量需要足够评分才能稳定估计；即便加了支持度下限，其候选也只有 193 部，命中空间比 ItemCF 的 3,000 部物品更窄。改进方向在第 7.2 节讨论（例如对物品因子加偏置项、或按支持度分档训练）。

4.6 新用户冷启动

4.6.1 方法与事实说明

冷启动算法解决“用户没有任何可用行为”时的推荐入口。它不引入新模型，而是把已有两路结果按用户状态路由：若用户存在至少一条正反馈历史（rating ≥ 4.0），则调用 Content 给出个性化结果，召回来源标记为 cold_start_content；若没有，则调用 Popular 给出全站正反馈热门榜，召回来源标记为 cold_start_popular。两条路径的输出都会被重写为冷启动算法自己的身份（algorithm = cold_start、model_version = coldstart-popular-content-v1），但保留来源标记，便于前端显示“冷启动 · 正反馈热度”或“冷启动 · 类型画像”。

过滤与回退：同样剔除已评分与已收藏电影；两路子算法的空结果会原样传递（用户无历史且热门榜被过滤完时返回 0 条），不使用随机结果填充。评价模式下使用 train.csv 训练的两个子模型，模型版本为 coldstart-popular-content-eval-v1。首页第一

栏即冷启动入口：注册后尚未评分时展示热门榜并附说明，评分之后自动切换为类型画像结果。

表 4-6 4.6 新用户冷启动

记录项	输入与数据范围	核心公式与符号定义	关键参数与取值	过滤与回退规则	实现文件与提交	运行与验证证据
ColdStart	用户正反馈数量（0 或无）；Popular 与 Content 结果	无正反馈 → Popular 榜；有正反馈 → Content 余弦排序	positive_rating_threshold = 4.0；max_recommendations = 10	剔除已评分与收藏；子算法空结果原样返回，不填充随机结果	recommender/student/cold_start.py；models/business/cold-start-v1/cold-start.json（第 2 次提交）	零行为账号返回 10 条 popular 来源结果；评分后切换为 content 来源；首页截图与接口响应一致

4.6.2 证据与分析

冷启动策略的效果可以从两个方向核对。零行为方向：新注册账号在首页与实验室中获得 10 条结果，全部来自全站正反馈热门榜，前端标注“非个性化”；有少量行为方向：账号给出 1—3 条正反馈后，结果切换为类型画像路径，内容相似度成为主要依据。冷启动 $k = 0$ 分区上 Popular 的 Precision@10 为 0.3369，说明这条兜底路径确实能把相关电影排到前面；而在 $k \geq 1$ 分区上 Content 与 ItemCF 的表现更好，

说明“先兜底、再个性化”的两段式顺序是合理的。

限制：冷启动阶段没有交互数据，无法做个性化排序；类型画像的区分度有限（第 4.3 节）；策略切换阈值目前是“至少一条正反馈”，没有做更细的探索（例如按行为数量或多样性分级）。这些都记录在 7.2 节的改进方向中。

4.7 扩展模块设计

4.7.1 方法与事实说明

扩展模块共四项，全部真实运行。第一项 S1 加权 RRF 融合：把多路召回按名次合并，第 s 路第 rank 名计 $1/(k + rank)$，$k = 60$，乘以该路权重后求和；默认来源为 ItemCF（权重 6）与 ALS（权重 1），权重与来源都可由请求参数覆盖。融合响应对每个来源给出候选数、入榜数、最佳名次，并统计被剔除的已交互条目（filtered_seen）与目录外条目（filtered_out_of_catalog）。为让每路提供足够素材，Provider 的返回上限放宽到 50 条，融合按 $\max(20, \min(50, limit \times 3))$ 请求深度。

第二项 S2 自然语言助手：用户在界面输入问题，后端把问题与工具声明发给兼容 OpenAI 协议的模型，模型只能从四个只读工具中选择调用：search_movies（按类型、年份区间检索真实目录）、similar_movies（读 ItemCF 工件的相似邻居）、recommend_movies（调用已注册的推荐算法）、my_history（读取当前账号或历史用户的真实评分与收藏）。模型返回的文本只用于组织回答，前端展示的电影卡片全部来自工具结果；回答中出现的、工具结果里不存在的片名会被记录为 unverified_mentions 并告警，被拒绝的工具调用（类型不存在、ID 不存在、参数越界）也会记入 trace，绝不静默降级。外部服务不可用时返回 503 ASSISTANT_UNAVAILABLE 并给出原因，不生成虚假结果。

第三项 S3 实时行为与短期兴趣：评分、收藏与取消收藏在数据库事务提交后向 Kafka 主题 cinelab.behavior.business 发送事件（事件含 schema、事件 ID、主体键、电影 ID、类型列表与时间戳，生产端尽力而为并带冷却）。独立消费进程按主体键维护类型兴趣权重：评分 ≥ 4.0 加 $(rating - 3.0)/2$，评分 ≤ 2.0 减 $(3.0 - rating)/2$，收藏加 1.5，取消收藏减 1.5，并随半衰期 30 分钟指数衰减（$factor = 0.5^{(\Delta t / 1800)}$）；状态写入 interest.json，事件流水写入 events.jsonl。推荐融合接口读取该状态后对结果重排：$match = \min(1,$ 该片类型兴趣权重和 / 最强类型权重$)$，$score' = score \times (1 + \alpha \times match)$，$\alpha = 0.5$；状态超过 900 秒未更新视为 stale，此时不加重排并在响应中说明原因。

第四项创新点 A——短期兴趣自适应融合权重：在 S3 状态可用的前提下，先计算每路候选与当前短期兴趣的平均匹配度 affinity_s，再按 $w'_s = w_s \times (1 + \beta \times affinity_s)$ 调整该路权重（$\beta = 0.5$，可由环境变量覆盖），最后用调整后的权重做 RRF。响应中新增 adaptive 区块，逐路给出基础权重、亲和度、调整后权重与候选数，并说明是否生效及原因；请求参数 adaptive=0|1 用于前后对照，默认关闭以保证既有实验结果可复现。

表 4-7 4.7 扩展模块设计

记录项	输入与数据范围	核心公式与符号定义	关键参数与取值	过滤与回退规则	实现文件与提交	运行与验证证据
S1 融合	多路 Provider 候选（各上限 50）	score = Σ_s w_s / (k + rank_s)；k = 60	默认 itemcf 6、als 1；深度 max(20, min(50, limit×3))	去重；剔除已评分与收藏；短路与越界条目计数上报	recommender/student/rrf.py、backend/app/fusion.py（第 4 次提交）	候选池 57；权重实验 19 组；Precision@10 0.0558
S2 助手	问题文本；真实目录、ItemCF 工件、推荐接口、行为历史	模型选择工具 → 工具返回事实 → 模型组织回答	四工具白名单；最多 3 轮；问题 2—300 字；limit 1—10	工具结果为空则如实说明；拒绝越界调用；外部不可用返回 503	backend/app/llm.py、assistant.py（第 6 次提交）	四场景脚本通过；单次提问 4,706 tokens；片名预警生效
S3 实时行为	评分/收藏/取消收藏事件；Kafka 主题	兴趣权重按 (rating−3)/2 与 ±1.5 累加；衰减 0.5^(Δt/1800)	半衰期 30 分钟；α = 0.5；stale 阈值 900 秒	无状态或过期不重排并说明；消费	backend/app/behavior.py、interest.py、scripts/run_behavior_consumer.py（第 5 次提交）	兴趣生效后 356 从第 6 位降到第 9 位；四类场景通过

				端独立进程		
创新点 A	每路候选与短期兴趣状态的匹配度	affinity_s = mean(match)；w′_s = w_s × (1 + β × affinity_s)	β = 0.5（CINELAB_ADAPTIVE_BETA）	interest=0 或状态过期即不生效并给出原因	backend/app/interest.py、fusion.py（第 6 次提交）	默认权重 6.0→8.2529、1.0→1.4176；等权时 ALS 反超；名次顺序改变

4.7.2 证据与分析

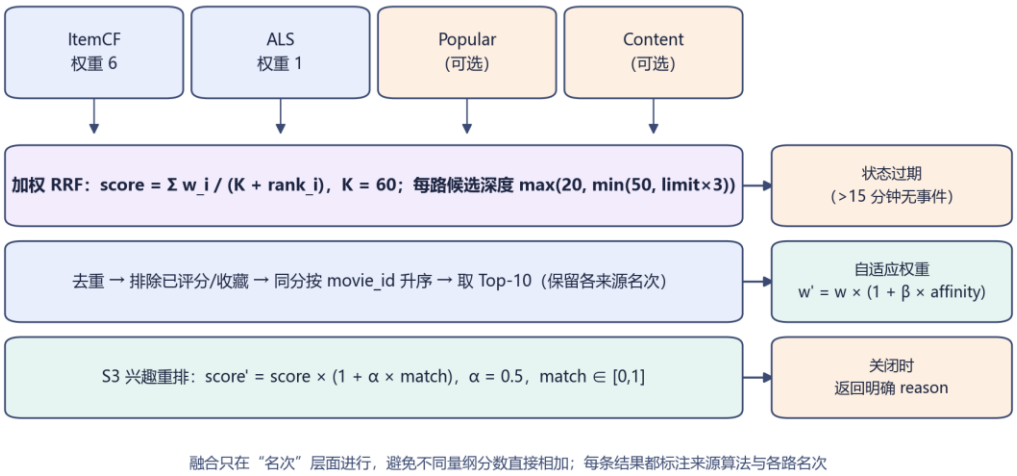


图 4-1 加权 RRF 融合流程（两路召回、名次计分、过滤与来源统计）

S1 的证据是 19 组权重对照实验：在验证集上，ItemCF 单路 0.0553、ItemCF×6＋ALS×1 为 0.0558（略优），等权与包含 Popular、Content 的组合都更低，说明“强路高权重、弱路低权重”比等权更合适；测试集上融合为 0.0522，低于单路 ItemCF 的 0.0517 之上（基本相同），说明融合的收益主要来自召回覆盖而非单纯的分数平均。

S2 的证据来自四类场景脚本：正常场景下模型调用 search_movies 返回 10 部动画片并组织回答，前端卡片全部带 source_tool；无结果场景下如实回答“没有找到符合条件的电影”并返回空列表；错误输入场景中不存在的类型调用被拒绝并记录；外部不可用场景下返回 503 且业务接口不受影响。护栏还捕获了两处模型“自由发挥”：回答里出现的片名若不在工具结果中会被列出（实测预警了 101 Dalmatians，该片确实不在检索结果里，属于正确预警）。

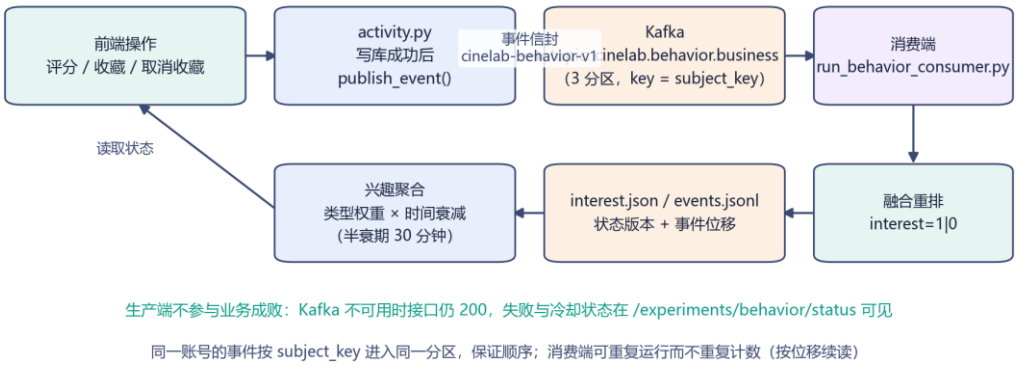


图 4-2 实时行为与短期兴趣链路（前端行为—Kafka—消费端—兴趣状态—重排）

S3 的证据是同一候选集合在 interest=0/1 两种设置下的 Top-10 对照：开启兴趣后，与当前兴趣匹配度低的电影 356 从第 6 位降至第 9 位，三部高匹配电影各上升一位；接口同时返回状态版本号与事件条数，便于核对消费端确实处理了新事件。创新点 A 的证据是三组权重对照：默认权重下 itemcf 6.0→8.2529、als 1.0→1.4176；等权（1:1）下 als 调整后权重超过 itemcf 并使名次改变；popular+itemcf 等权时 Top-10 顺序从 260,356,1270… 变为 260,1270,356…，说明自适应权重确实改变排序而不是仅仅改变数字。

四项扩展的共同限制是“扩展能力不进入课程契约”：融合、助手与兴趣重排都通过独立模块与独立接口提供，没有注册进 contracts.py 的 PROVIDERS，也没有修改冻结的业务表。好处是基础五算法的评价结果不受扩展影响，代价是扩展结果无法通过课程的 check_result 契约校验，只能由项目自建的集成检查与场景脚本覆盖。

5 系统实现与功能测试

5.1 系统运行环境

5.1.1 方法与事实说明

运行环境为 Windows 11 家庭版（24H2 及以上）+ WSL2 + Docker Desktop，容器以 linux/amd64 运行。课程提供三个离线镜像：cinelab-offline/api:1.3.0-a1（FastAPI 后端，Python 3.12.12）、cinelab-offline/web:1.3.0-a1（Nginx + 前端产物）、cinelab-offline/mysql:8.0.46-a1（数据库）。三个镜像通过脚本从 tar 包导入本机，compose 以项目名 cinelab 启动 web、api、db 三个容器；扩展模块再叠加 Kafka 容器（apache/kafka:3.9.0，KRaft 单节点）。前端静态资源由 web 容器的 Nginx 提供，

宿主通过 http://127.0.0.1:8080 访问。

宿主环境承担三项容器内无法完成的职责：前端构建（Node + Vite）、离线训练（Python 3.11.8 + Spark 3.5.3 + JDK 11）与离线评价脚本（宿主 Python 直接调用 recommender 包）。密钥与口令放在 deployment/private/ 目录（数据库口令、JWT 密钥、模型接口密钥），compose 以只读方式挂载进容器；该目录被 .gitignore 排除，不会进入仓库。数据包与模型目录同样以只读方式挂载，容器根文件系统只读，禁止权限提升。

启动命令分两层：基础服务使用 docker compose up -d；含扩展的完整环境使用三个文件叠加（compose.yaml + deployment/lab5/compose.lab5.yaml + deployment/lab6/compose.lab6.yaml），后两个文件分别补入 Kafka 与事件/兴趣环境变量、模型密钥挂载与自适应权重参数。停止与重启使用 compose stop/start；课程提供的“恢复教师原始镜像”脚本会覆盖本地派生镜像，现场演示前必须改用叠加文件启动，这一点写入了验证说明。

表 5-1 5.1 系统运行环境

项目	记录 1	记录 2	记录 3	记录 4
容器	cinelab-web-1（Nginx）	cinelab-api-1（FastAPI）	cinelab-db-1（MySQL 8.0.46）	cinelab-kafka-1（Kafka 3.9.0）
端口	127.0.0.1:8080	仅容器网络	仅容器网络	127.0.0.1:29092
挂载	前端产物（镜像内）	data 包只读、models 只读、posters 读写、密钥只读	mysql_data 卷	report/state 目录
验证	页面可访问	/health/ready = ready	八表行数一致	kafka-topics.sh 健康检查

5.1.2 证据与分析

环境验证证据：/health/ready 返回 status=ready 以及数据版本三元组（dataset/split/schema）与 manifest 摘要前缀；四个容器在检查期间均为 healthy；api 容器内可导入 httpx、sqlalchemy、fastapi，但无法导入 numpy、pandas 与 Java 运行时，这与课程镜像的构建清单一致，也解释了为什么算法推理全部用标准库实现、训练放在宿主完成。

环境搭建过程中遇到并修复的问题有三类。第一类是 WSL 子系统与 Docker 引擎的安装问题（在环境排查记录中单独说明）；第二类是编码问题：模型清单若包含非 ASCII 字符，评审端按系统默认编码读取时会解码失败，最终约定清单只写 ASCII，中文说明放在文档而不是清单里；第三类是网络限制：本机无法访问 Docker Hub，

课程的联网重建路径不可用，扩展镜像改为离线安装包与本地派生完成。

5.2 用户与电影业务实现

5.2.1 方法与事实说明

用户与电影业务由课程既有实现承载，本项目在其上完成了展示与排布优化。功能包括：注册与登录（用户名 3—64 位、口令 12—128 位，登录态为 HttpOnly Cookie，写操作带 CSRF 令牌与同源 Origin 校验）；电影库（按类型、年份、关键词筛选，分页每页 15 部，卡片显示海报、标题、年份与类型）；详情页（电影信息、评分表单、收藏按钮、我的评分回显）；个人中心（我的评分、我的收藏、账号信息）；冷启动引导页（说明“还没有行为时系统如何推荐”）。

行为接口的写入规则是：评分取值 0.5—5.0、步长 0.5，重复评分覆盖旧值；收藏与取消收藏成对；两者都要求已登录，且都会在事务提交后发布一条 Kafka 行为事件（第 4.7 节）。为避免前端误用，接口层对非法取值直接返回 422 VALIDATION_ERROR，对不存在的电影返回 404 MOVIE_NOT_FOUND，并且失败的写入不会产生事件，这一点由场景脚本断言。

前端改造集中在三处：海报（新增按需抓取与本地缓存的海报接口，卡片与详情页统一使用）、主题（由深色改为浅色，变量集中在 theme.css，页面样式追加在 style.css，避免改动课程既有样式段）、排布（压缩页面留白、电影库由每页 12 部改为 15 部以填满 5 列网格、推荐卡片由双列改单列、长页面分区块提高信息密度）。改造过程中保留了既有的组件接口与路由结构，没有修改后端业务逻辑。

表 5-2 5.2 用户与电影业务实现

项目	记录 1	记录 2	记录 3	记录 4
功能	注册/登录	电影库与筛选	评分与收藏	个人中心
接口	POST /auth/register 、/auth/login、GET /auth/csrf	GET /movies 、GET /movies/{id} 、GET /genres	PUT/GET /me/ratings 、PUT/DELETE/GET /me/favorites	GET /me、GET /me/ratings 、GET /me/favorites
校验	用户名与口令长度、口令强度	分页 1—100、筛选枚举	0.5—5.0 步长 0.5；ID 必须存在	会话有效性与 CSRF
证据	fig-login.png 、fig-register.png	fig-movies.png 、fig-movies-pagination.png	fig-detail.png 、fig-profile.png	接口断言与截图

5.2.2 证据与分析

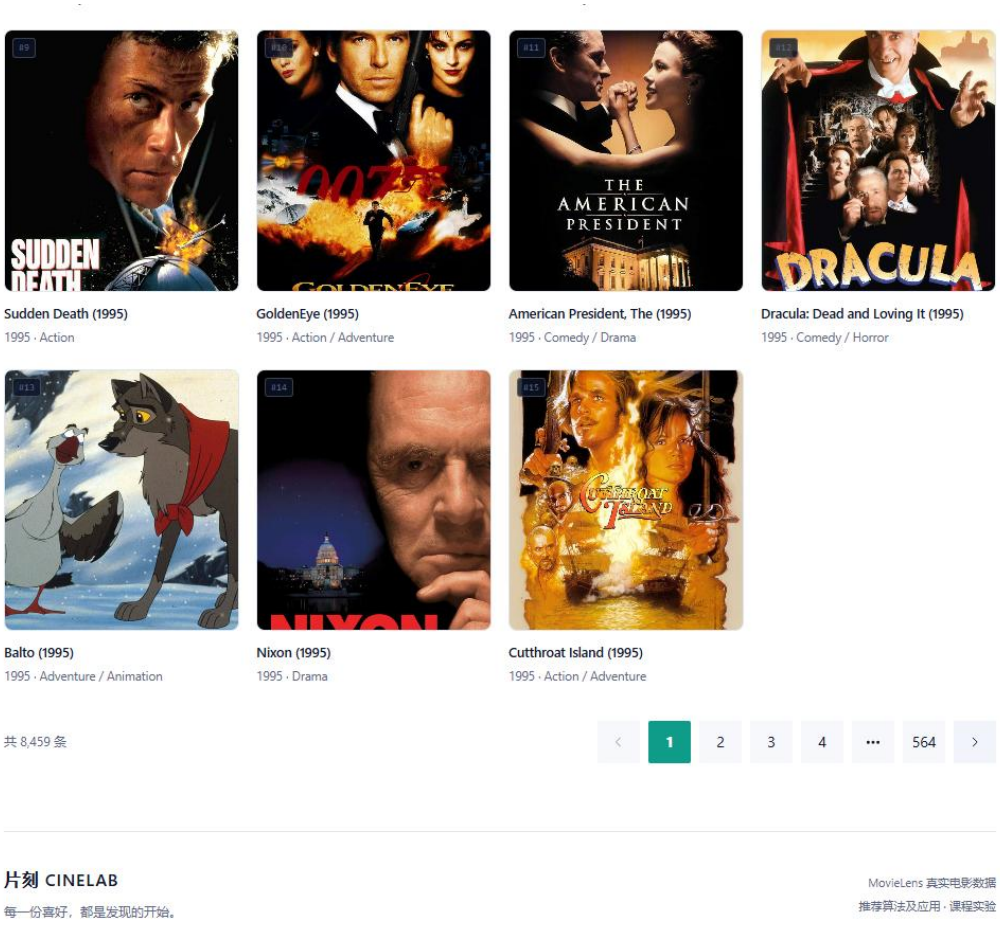


图 5-1 电影库页面（每页 15 部、5 列网格、筛选与分页）

业务功能的验证包含正向与负向两类。正向：注册后登录、评分 1→5.0、13→4.5、48→2.0、收藏电影 1，接口全部返回 200，个人中心显示一致；负向：评分 4.3 与 6.0 返回 422、不存在的电影 ID 返回 404、未登录写入返回 401、缺少 CSRF 返回 403。负向用例同时断言“失败不产生行为事件”，避免前端误操作污染实时兴趣状态。

排布优化的效果用页面高度衡量：电影库从 2,151 px 降到 1,702 px，个人中心从 1,618 px 降到 1,000 px，首页从 2,658 px 降到 2,186 px，推荐实验室从 10,178 px 降到 5,705 px。这些数字来自浏览器实测（1440 × 1000 视口，整页高度），不是估算。

5.3 推荐算法实验室实现

5.3.1 方法与事实说明

推荐算法实验室是一个页面承载多种验证能力的界面：左侧控制区可选择数据来源（我的账号 / 历史用户）、算法（五种）、结果条数，并可切换“对照模式”同时查看两种算法的结果；右侧结果区分区块展示算法结果卡片（海报、名次、类型、匹配

度、来源标签）、融合面板（来源权重、候选统计、兴趣区块、自适应权重）、离线评价表（验证集、测试集与冷启动分区）、实时行为面板（生产端与消费端状态、最近事件）与自然语言助手面板（问题输入、工具轨迹、回答与卡片、token 用量）。

页面与接口的对应关系是固定的：算法结果读取 /me/recommendations/{algorithm} 或历史用户同构接口；融合面板读取 /me/recommendations/fusion 与 /experiments/fusion/sources；评价表读取构建期发布的 lab3-metrics.json；行为面板读取 /me/behavior/interest 与 /experiments/behavior/status；助手面板读取 /me/assistant/query。其中 /experiments/evaluations 按课程契约返回 501，页面保留该契约提示并显示构建期指标，避免把“在线计算指标”写进界面。

为保证可复现，实验室页面提供“预设”入口（例如融合的默认权重、等权对照、自适应开关），点击后把参数写入地址栏查询串，评审可以直接复制链接复现同一组结果。所有结果卡片都显示模型版本与召回来源，便于与文档中的表格逐条对照。

表 5-3 5.3 推荐算法实验室实现

项目	记录 1	记录 2	记录 3	记录 4
面板	算法结果	融合与自适应	离线评价表	助手面板
接口	/me/recommendations/{algorithm}	/me/recommendations/fusion	构建期 JSON（lab3-metrics.json）	/me/assistant/query
关键展示	名次、类型、匹配度、来源	各路权重、候选数、兴趣区块	验证集/测试集/冷启动分区指标	工具轨迹、回答、卡片、tokens
验收截图	fig-lab-content-ml14.png 等五张	fig-fusion-panel.png、fig-fusion-adaptive.png	fig-evaluation-validation.png	fig-assistant.png 、fig-assistant-accuracy.png

5.3.2 证据与分析



图 5-2 推荐实验室：历史用户 ml:14 的 Content 结果（Top-10 与来源标签）

实验室验证覆盖了五种算法 × 两类数据来源：ml:14 与 ml:55 各五种算法均返回 10 条且与各自已评分集合零交集；新注册账号在 content、itemcf、als 上返回 0 条（如实空结果）、在 cold_start 上返回 10 条热门榜结果；实验室对照模式同时请求两路并排展示，用于说明"内容相似与行为共现的差异"（同一用户下两路 Top-10 的差异在文档第 6.4 节给出对照表）。

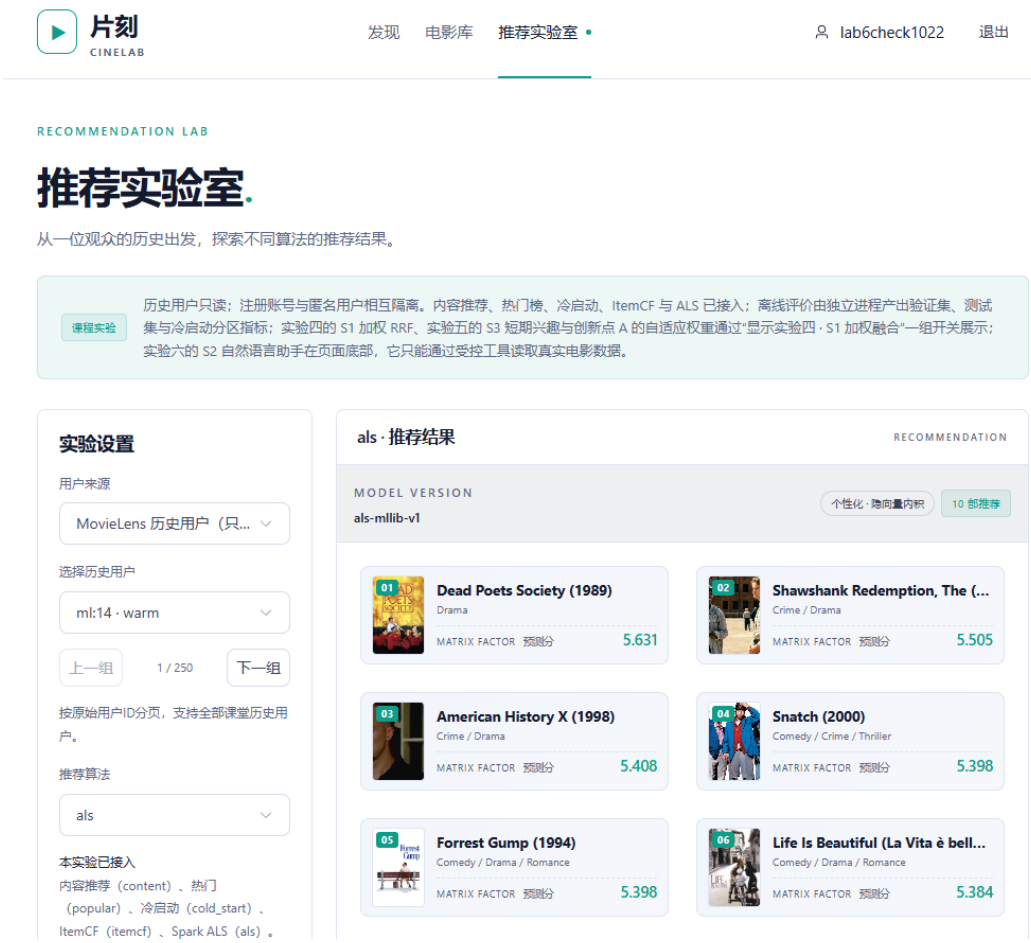


图 5-3 推荐实验室：ml:14 的 ALS 结果（隐向量内积分数与模型版本）

验证集 · 调参与方案选择 4,209 位固定评价用户

正反馈阈值 4.0（评分 ≥ 4.0），Top-10；候选已排除该用户在 train 中的评分，无结果的用户仍计入宏平均分母。

算法	Precision@10	Recall@10	有效用户	空结果
popular	0.0342	0.0578	4209	0
content	0.0041	0.0075	4209	0
itemcf	0.0553	0.0978	4209	0
als	0.0414	0.0672	4209	1
cold_start	0.0041	0.0075	4209	0

图 5-4 实验室中的离线评价表（验证集五个算法的 Precision@10 与 Recall@10）

5.4 推荐接口与系统集成

5.4.1 方法与事实说明

推荐接口的调用链在服务端固定为六步：路由解析参数并构造RecommendationRequest；契约层 recommend() 按 Algorithm 枚举查找已注册Provider（未注册则返回 501 ALGORITHM_NOT_IMPLEMENTED）；校验请求（limit、runtime_mode、数据版本）；校验模型清单（算法身份、版本正则、数据版本三元组、训练文件摘要、工件摘要）；调用 Provider 得到 RecommendationResult（其间由业务历史读取器提供“已评分与收藏”集合）；校验结果 DTO（字段不多不少、rank 连续、movie_id 在目录内、去重、同分按数值升序、身份字段一致），任一步失败返回结构化错误。

前端集成通过统一封装的请求函数完成：GET 请求直接调用；写操作先获取 CSRF 令牌再提交，并携带同源 Origin；错误按 JSON 信封解析出 code 与 message，页面据此显示“未实现（501）”“数据不可用（503）”“校验失败（422）”等明确状态。列表与详情使用并发请求（详情页一次性拉取所需电影信息），实验室页面按区块懒加载，避免一次请求返回全部面板数据。

集成过程中的一个真实缺陷记录在此：融合路由最初注册在实验路由之后，导致 /me/recommendations/fusion 被 /me/recommendations/{algorithm} 截获并返回 501。修复方式是把融合路由提前注册，并在代码注释中说明原因。这类“路径优先级”问题在集成阶段容易漏掉，因此被写入集成检查脚本（断言 fusion 端点返回 200 且包含 sources 字段）。

表 5-4 5.4 推荐接口与系统集成

项目	记录 1	记录 2	记录 3	记录 4
阶段	参数解析与请求构造	契约与模型校验	Provider 计算	结果校验与返回
失败示例	limit 越界 → 422	未注册算法 → 501；清单不合法 → 500	数据身份不符 → DATA_IDENTITY_INVALID	去重/名次/同分违规 → 结果校验失败
证据	实验室非法参数用例	注册表接口与 501 契约	集成检查断言	集成检查 31 项
代码位	backend/app/experiments.py、fusion.py	recommender/contracts.py:recommend()	recommender/student/*.py	recommender/validation.py:check_result()

置				

5.4.2 证据与分析

接口集成的验证以集成检查脚本为主：脚本以真实账号登录后依次请求 31 组断言，覆盖数据身份、注册表、写入与读取、五个算法的输出约束、历史用户隔离、融合字段、兴趣状态、评价表计数、助手工具结果来源、前端产物标记等。整轮检查在真实容器上运行，任何一项失败都以非零退出码结束，因此可以直接作为回归测试使用。

另一类证据是契约校验命令：对每个模型执行 check model 校验清单与工件摘要，对每个分区执行 check predictions 与 check evaluation-result 校验离线评价文件；业务侧对五个算法执行 check selfcheck，确认接口在真实 HTTP 服务上返回合法 DTO。这些命令由课程工具提供，通过即说明实现满足契约，而不是“看起来能跑”。

5.5 系统功能测试

5.5.1 方法与事实说明

测试分四层。第一层是契约校验（课程工具）：模型清单校验、评价输入校验、预测文件与结果文件校验、业务自检；第二层是接口集成检查：31 项断言覆盖端到端数据流；第三层是场景脚本：助手四场景、行为四场景、自适应三组对照；第四层是前端验收：逐页截图与关键交互（登录、筛选、评分、收藏、算法切换、融合预设、助手提问）人工核对。四层测试的产物都写入 reports 目录并可重复执行。

断言设计遵循三条原则：可判定（只断言确切事实，如“交集为空”“rank 从 1 连续”）、可复现（命令固定、参数固定、数据版本固定）、可定位（失败时输出差异对象，如缺失的字段名、冲突的 movie_id）。脚本对每个场景输出 passed 与 failed_checks，便于快速定位失败环节。

测试数据也做了隔离：集成检查使用独立账号（lab6check 系列），场景脚本使用各自的前缀（lab5demo、lab6adapt），避免相互污染；历史用户只读，不写入任何行为；Kafka 场景测试会显式启停消费端并在报告中记录状态版本变化。

- 契约层：模型清单、评价输入、预测与结果文件、业务自检。
- 集成层：31 项断言，覆盖身份、注册表、写入、五算法、历史用户、融合、兴趣、助手与前端产物。
- 场景层：助手四场景、行为四场景（含 Kafka 停机）、自适应三组权重对照。
- 验收层：逐页截图与关键交互核对，截图随报告提交。

5.5.2 证据与分析

测试结果：集成检查 31 项全部通过（0 失败）；助手四场景全部 passed；行为四场景（正常、空态、错误输入、Kafka 停机）全部 passed，其中停机场景验证了“业务不中断、失败可见、推荐仍可用”；自适应三组对照全部通过，权重与名次变化均有记录；契约校验命令全部通过。所有报告保存在 reports 目录（lab3、lab4、lab5、lab6 子目录），并在文档中给出复现命令。

测试过程中发现并修复了三个真实缺陷：其一，助手调用外部模型时缺少 tool_call_id 字段，模型服务返回 422（修复后工具调用链正常）；其二，热门榜排序在分数相同时未按 movie_id 升序，违反课程契约对并列的要求（修复后集成检查新增该断言）；其三，模型清单包含中文字符导致读取端按系统编码解码失败（统一为 ASCII 清单并增加构建期检查）。这三个缺陷说明测试的价值不在“证明没问题”，而在于把隐藏的假设暴露出来。



图 5-5 推荐实验室整体布局（控制区、结果区、评价表与扩展面板）

前端验收中还总结了一个操作教训：Element Plus 的复选框真实 input 被隐藏，自动化脚本若直接点击会切换两次而恢复原状，导致截图与预期不符。最终改为读取状态后按需点击外层标签，并再次核对截图内容，避免“看起来拍到了”却与设置不一致。

5.6 扩展功能实现

5.6.1 方法与事实说明

扩展功能的实现在部署层与代码层同时落地。部署层新增两个覆盖文件：deployment/lab5/compose.lab5.yaml 增加 Kafka 服务（KRaft 单节点、容器内 kafka:9092、宿主 127.0.0.1:29092）、为 api 注入事件主题与兴趣状态环境变量并挂载状态目录，同时用本地派生镜像（在课程镜像上安装离线 kafka-python 包）替换 api；deployment/lab6/compose.lab6.yaml 追加模型密钥目录只读挂载、模型接口与自适应权重环境变量、海报目录挂载，并用本地派生镜像替换 api 与 web。两个目录各自附带 README，说明镜像来源、构建命令与变量含义。

代码层新增七个后端模块：behavior.py（事件生产，尽力而为、带冷却与失败计数）、interest.py（状态载入、匹配度计算、重排与自适应权重）、behavior_api.py（兴趣与行为状态接口）、llm.py（外部模型客户端，支持真实与占位两种模式）、assistant.py（工具白名单、参数校验、轮次控制与结果组装）、posters.py（海报抓取、缓存、命中统计与回退）、fusion.py（融合与自适应权重，改造既有实现）。前端新增两个组件（FusionPanel、AssistantPanel）并改造若干页面（首页、实验室、电影库、详情、个人中心）与主题样式。

运维与脚本层新增九个脚本：模型构建（三个）、离线评价运行与指标发布、行为消费端、行为场景、助手场景、自适应对照、海报预热、集成检查。所有脚本都以“可重复执行 + 输出 JSON 报告”的方式编写，便于课程验收时直接运行并查看结果。

表 5-5 5.6 扩展功能实现

项目	记录 1	记录 2	记录 3	记录 4
扩展	S1 融合	S2 助手	S3 实时行为	创新点 A 自适应
部署文件	deployment/lab5、lab6 compose	lab6 密钥挂载	lab5 Kafka 服务	lab6 环境变量
代码文件	recommender/student/rrf.py、backend/app/fusion.py	backend/app/llm.py、assistant.py	behavior.py、interest.py、behavior_api.py	interest.py（adaptive_weights）、fusion.py
验证脚	scripts/eval_lab4_fusion.py	scripts/lab6_assistant_scenarios.py	scripts/lab5_scenarios.py	scripts/lab6_adaptive_check.py

本				

5.6.2 证据与分析

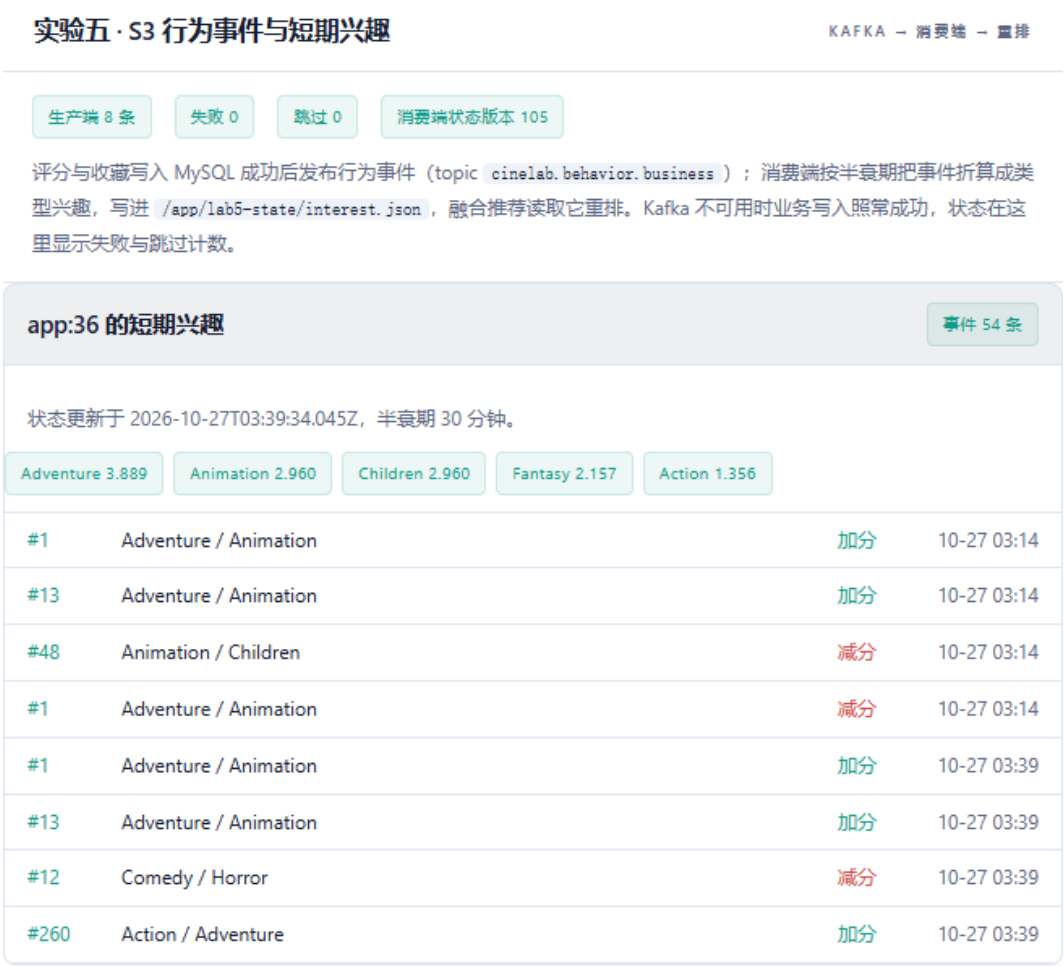


图 5-6 实时行为面板（生产端与消费端状态、最近事件与状态版本）

扩展功能的端到端证据由截图与脚本共同构成：行为面板显示生产端 sent/failed/skipped 计数与消费端状态版本，评分后版本递增；融合面板显示两路基础权重、候选数、兴趣区块（匹配度与加权倍数）与自适应权重；助手面板显示工具轨迹（工具名、是否成功、命中条数）、回答文本、电影卡片与 token 用量；首页第一栏在冷启动与个性化之间按行为状态切换。



图 5-7 助手面板（推荐算法与离线准确率随回答一并展示）



图 5-7（续 1） 助手面板（推荐算法与离线准确率随回答一并展示）

部署证据：叠加三个 compose 文件启动后四个容器全部 healthy；执行 docker compose stop/start 后状态可恢复；海报缓存目录在宿主可见并随抓取增长（检查时已

缓存数千部电影的海报，未命中的电影由矢量海报兜底）。运行限制如实说明：课程提供的“恢复原始镜像”脚本会覆盖本地派生镜像，演示与验收必须使用叠加文件启动。

6 推荐算法实验与结果分析

6.1 实验环境与数据设置

6.1.1 方法与事实说明

实验环境分为两处：离线训练与评价在宿主 Windows 11 上完成（Python 3.11.8、Spark 3.5.3 本地模式、JDK 11、numpy 2.4.3、pandas 2.2.2、scipy 1.17.1），在线推理在容器内完成（Python 3.12.12，仅标准库）。这样安排的原因是课程离线镜像不包含数据科学依赖与 Java 运行时，而模型工件（JSON）可以在两处之间安全传递：宿主训练 → 写入 models 目录 → 容器只读挂载 → 标准库推理。

数据设置遵循课程冻结协议。训练使用 evaluation/train.csv（583,502 行），验证使用 validation.csv（70,366 行、有效用户 4,209 位），测试使用 test.csv（67,792 行、有效用户 4,198 位），冷启动使用 cold-k0/1/5/10 四个分区（各 499 位有效用户）。正反馈阈值为 rating ≥ 4.0；候选集合按分区累计排除该用户已评分物品（测试集同时排除训练与验证）；Precision@10 的分母固定为 10，Recall@10 的分母为该用户候选内正反馈物品总数；对固定用户集合取宏平均，并列时按 movie_id 数值升序。

调参纪律：所有参数选择只在验证集上进行，测试集只报告一次最终结果，不在测试集上反复调整。验证集用于三项对照实验：ALS 的隐向量维度与正则系数网格（9 组）、ALS 的候选支持度下限网格（5 组）、融合权重网格（19 组）；每一项都在报告中给出完整网格与选择依据，而不是只给最优值。

表 6-1 6.1 实验环境与数据设置

项目	记录 1	记录 2	记录 3	记录 4
训练环境	宿主 Python 3.11.8	Spark 3.5.3 本地模式	JDK 11	numpy / pandas / scipy
推理环境	容器 Python 3.12.12	仅标准库	模型工件只读挂载	MySQL 8.0.46 业务库
数据分区	train 583,502	validation 70,366（4,209 人）	test 67,792（4,198 人）	cold-k0/1/5/10（各 499 人）
调参纪律	只在验证集调参	测试集只报一次	网格全量记录	并列按 ID 升序

6.1.2 证据与分析

数据设置的证据是两次运行的一致性：评价脚本为每个分区生成 task_sha256（任务描述摘要）与 user_ids_sha256（用户列表摘要），同一分区重复执行时两个摘要不

变；模型清单中记录训练文件摘要，check_model 会核对它与数据包中的实际文件是否一致。这两项共同保证“实验是在同一份数据上做的”。

限制说明：本实验只使用课程给定的单次时间划分，没有做多折交叉验证；因此算法之间在小数第三位上的差距不应被解释为稳定优势。这一点在第 6.6 节作为效度威胁再次说明。

6.2 评价指标

6.2.1 方法与事实说明

Precision@10 定义为前 10 条中正反馈电影数量除以 10。分母固定为 10 的含义是：即使某用户候选内可推荐电影不足 10 条（例如冷启动分区），也按 10 计，从而不让“候选少”自动抬高指标。Recall@10 定义为前 10 条中正反馈电影数量除以该用户候选内正反馈电影总数，衡量覆盖能力；两个指标一起看可以避免“只推热门”与“只推长尾”两种极端。

宏平均的做法是先对每位有效用户计算指标，再对用户取平均，使每位用户权重相同，不会被行为多的用户主导。并列规则（同分按 movie_id 数值升序）保证同一候选集合在任何实现下得到完全相同的排序，从而让指标可复现。评价结果以 JSON 文件发布（reports/lab3/summary.json 与前端构建期文件），每条记录包含算法、分区、用户数、空结果用户数、Precision@10、Recall@10、模型版本与耗时。

除主指标外，本项目还记录空结果用户数（empty_users）。该字段用于区分“算法不适用”与“算法表现差”：例如冷启动 k = 0 分区上 Content、ItemCF、ALS 的空结果用户数都是 499（没有历史可用、支持度不足），而 Popular 与 ColdStart 在该分区有结果；不记录空结果数会把“没有输出”误算成“输出质量低”。

表 6-2 6.2 评价指标

项目	记录 1	记录 2	记录 3	记录 4
指标	Precision@10	Recall@10	空结果用户数	耗时（秒）
定义	命中数 / 10（分母固定）	命中数 / 候选内正反馈总数	返回 0 条的用户数	分区整体运行时间
作用	排序准确度	覆盖能力	区分不适用与表现差	成本对照
证据文件	reports/lab3/summary.json	同上	同上（ empty_users 字段）	同上（ seconds 字段）

6.2.2 证据与分析

指标口径的统一性在实验中被验证：对同一分区重复运行评价脚本，Precision@10

与 Recall@10 完全一致（确定性实现 + 固定并列规则）；模型清单中记录的最终验证集指标（ALS：0.041388 / 0.067163，4,209 位用户、1 位空结果）与评价报告一致，这一致性由集成检查脚本自动核对。

6.3 基础算法对比实验

6.3.1 方法与事实说明

基础算法对比实验在验证集与测试集上执行同一套流程：冻结分区 → 逐算法生成预测（覆盖全部有效用户，无法生成的用户提交空列表）→ 官方校验预测文件 → 计算指标 → 写入报告。五个算法是 Popular、Content、ItemCF、ALS 与 ColdStart；另外报告融合方案（ItemCF×6 + ALS×1）作为对照，说明多路召回合并后的效果。所有算法都使用同一份已交互集合做过滤，因此差异来自排序而不是过滤。

对比实验的记录方式是“算法 × 分区 × 指标”的二维表，验证集与测试集分开呈现；表下给出三条以上的数据解释（为什么某算法更好、差异是否显著、与假设是否一致），并明确写出“本协议下”的限定，避免把单次划分的差距说成普遍规律。

表 6-3 6.3 基础算法对比实验

项目	Popular	Content	ItemCF
验证集 Precision@10	0.0342	0.0041	0.0553
验证集 Recall@10	0.0578	0.0075	0.0978
测试集 Precision@10	0.0312	0.0037	0.0517
测试集 Recall@10	0.0516	0.0070	0.0902

6.3.2 证据与分析

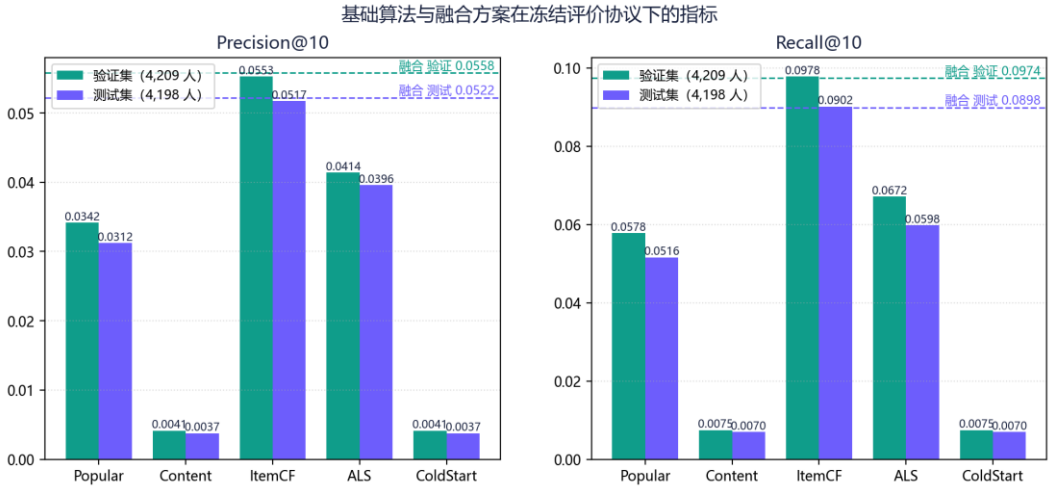


图 6-1 五类算法与融合方案的 Precision@10 与 Recall@10 对比（验证集 4,209 人、测试集 4,198 人）

结果排序（验证集 Precision@10）：ItemCF 0.0553 > ALS 0.0414 > Popular 0.0342 > Content 0.0041 = ColdStart 0.0041；融合方案为 0.0558，略高于单路 ItemCF；测试集

上同一排序保持（ItemCF 0.0517 > ALS 0.0396 > Popular 0.0312 > Content 0.0037），融合为 0.0522。两个分区的相对关系一致，说明排序结果不是验证集上的偶然。

三条数据解释：第一，ItemCF 的领先来自共现信号的密度（训练集每位暖用户平均约 130 条评分），而 Top-10 命中率正好衡量“共现邻居是否落在相关集合里”；第二，ALS 的隐向量需要更多评分才能稳定估计，且其候选被支持度下限收缩到 193 部，命中空间更窄，因此低一档；第三，Content 与 ColdStart 相同（0.0041）是因为在这一分区上冷启动用户的画像只由少量正反馈构成，路由到内容路径，两者输出一致——这也解释了为什么冷启动算法在验证集上的指标与内容算法相同。

限制：验证集与测试集上算法之间的差距处在同一量级（0.03—0.05），没有做置信区间估计与显著性检验；因此报告只陈述“在本协议、本数据版本下的相对关系”。

6.4 不同用户的推荐结果分析

6.4.1 方法与事实说明

用户维度的分析比较两类对象。第一类是历史用户（行为冻结、可与他人复现）：ml:14（15 条历史评分）与 ml:55（19 条历史评分），分别查看五种算法的 Top-10 与交集约束；第二类是真实注册账号：零行为新账号（验证冷启动兜底）、少量行为账号（验证内容画像切换）与持续行为账号（验证实时兴趣与自适应权重）。比较的判据有三条：结果是否与已评分集合零交集、不同算法的结果是否真的不同、行为变化后结果是否随之变化。

表 6-4 6.4 不同用户的推荐结果分析

项目	记录 1	记录 2	记录 3	记录 4
对象	ml:14（15 条历史）	ml:55（19 条历史）	零行为新账号	持续行为账号
Content top3	5 / 18 / 19	5388 / 7445 / 8860	0 条（如实空结果）	随评分变化
ItemCF top3	2502 / 1079 / 3039	2000 / 2194 / 1610	0 条	随评分变化
ALS top3	1246 / 318 / 2329	593 / 318 / 110	0 条	随评分变化

6.4.2 证据与分析

历史用户的对照显示“内容相似”与“行为共现”确实给出不同的电影：ml:14 的 Content 前三为《玩具总动员》系列与其他动画/家庭片，ItemCF 前三则是共现邻居（2502、1079、3039）；ml:55 的 Content 前三为 5388、7445、8860，ItemCF 为 2000、2194、1610。两路结果的差异正是第 4.7 节融合实验的前提——如果各路输出高度重合，融合就没有意义。

冷启动分区的趋势与直觉一致：Popular 在 k = 0 时最高（0.3369），随历史增多而缓慢下降（k = 10 时 0.2697）；Content 在 k = 1 时只有 0.0238，随历史增多升到 0.0287—0.0305；ItemCF 在 k = 1 时 0.1858，k = 5 升至 0.2750，k = 10 为 0.2667；ALS 从 k = 1 的 0.1379 升到 k = 10 的 0.2086。这说明“无历史靠热门、少量历史靠共现、更多历史靠隐向量”的分工在数据上是有依据的；同时也说明课程把冷启动单独划分分区是必要的，否则这些差异会被整体指标掩盖。

账号侧的证据来自真实操作：零行为账号在 Content、ItemCF、ALS 上得到 0 条，在 ColdStart 上得到 10 条热门榜结果；给出 3 条正反馈后，同一接口开始返回类型画像结果；开启实时兴趣后，Top-10 顺序随新事件变化（第 6.5 节给出对照）。这三步构成“新用户 → 少量行为 → 持续行为”的完整链路验证。

6.5 关键参数影响分析

6.5.1 方法与事实说明

关键参数影响分析包含四组实验。第一组是 ALS 的隐向量维度与正则系数：在 maxIter = 12 下遍历 rank ∈ {8, 16, 32} × regParam ∈ {0.05, 0.1, 0.2} 共 9 组，用验证集宏平均 Precision@10 选择（并列时看 Recall@10，再选更小的正则与维度）；第二组是 ALS 的候选支持度下限：在选定维度与正则后遍历 {0, 100, 200, 300, 500}，观察候选规模与指标的关系；第三组是融合权重：固定候选集合只改各路权重，遍历 19 组配置；第四组是实时兴趣与自适应权重：同一候选集合下切换 interest 与 adaptive 参数，观察名次变化与权重变化。

参数记录的完整度是本节的重点：每一组实验都保留完整网格与每一格的指标、耗时与候选数，写入模型清单（ALS 的 tuning 字段）或报告文件（融合与自适应）。这样评审可以核对“为什么选这个参数”，而不是只看到一个结果。

表 6-5 6.5 关键参数影响分析

项目	记录 1	记录 2	记录 3	记录 4
实验	ALS 维度与正则	ALS 支持度下限	融合权重	兴趣与自适应
网格规模	9 组（rank × regParam）	5 组（0—500）	19 组权重配置	3 组预设 + 开关对照
选择	验证集 Precision@	验证集 Precision@10	验证集 Precision@10	名次变化与权重可解释性

依据	10			
记录位置	als 清单 tuning.report	als 清单 tuning.support_floor_report	reports/lab4/fusion-tuning.json	reports/lab6/adaptive-weights.json

6.5.2 证据与分析

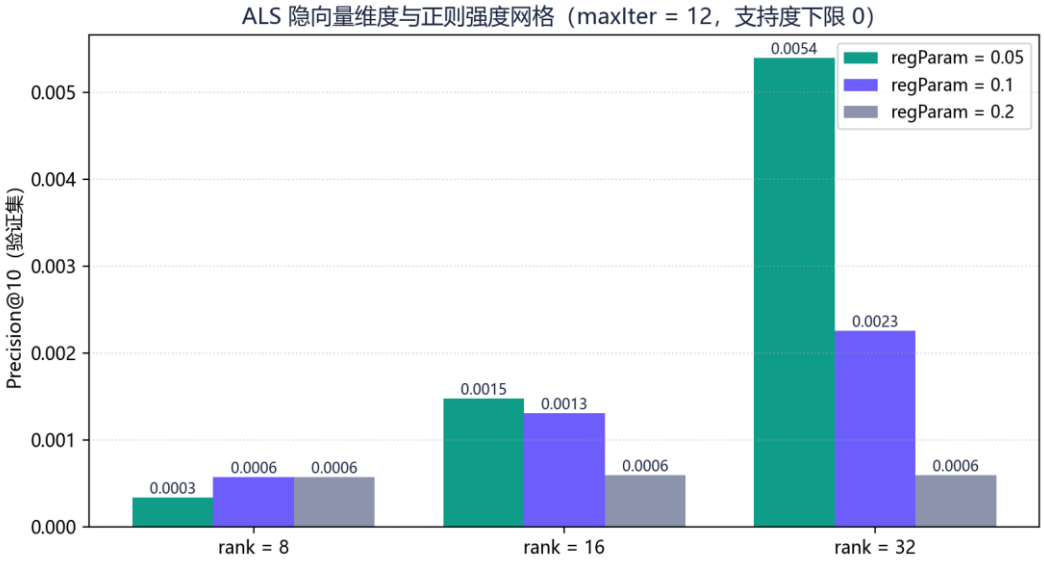


图 6-2 ALS 的维度与正则网格（验证集 Precision@10，9 组配置）

ALS 网格结果：rank = 8 时 Precision@10 为 0.0003—0.0006；rank = 16 为 0.0006—0.0015；rank = 32 为 0.0006—0.0054，其中 rank = 32、regParam = 0.05 最优（0.0054 / Recall 0.0113），因此选定 rank = 32、regParam = 0.05。维度的影响远大于正则：同一维度下调整 regParam 带来的变化不足一个数量级，而维度从 8 提升到 32 使指标提高约 10 倍，说明本数据集的偏好差异需要更高维表达。

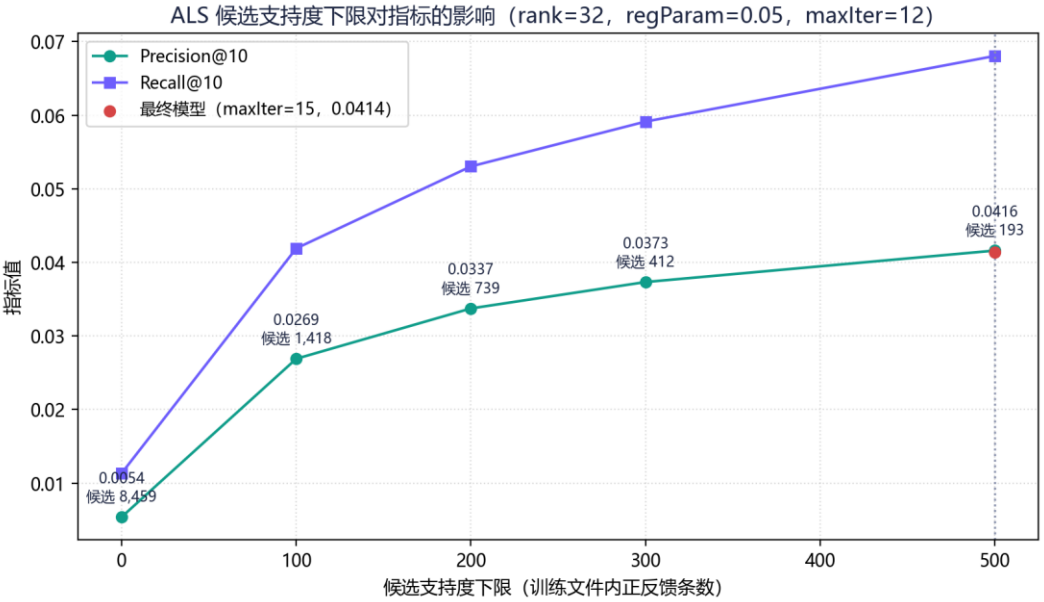


图 6-3 ALS 候选支持度下限的影响（候选规模与 Precision@10）

支持度下限实验的结果是本节最显著的一组：下限 0（全目录 8,459 部候选）Precision@10 为 0.0054；下限 100（1,418 部）为 0.0269；200（739 部）为 0.0337；300（412 部）为 0.0373；500（193 部）为 0.0416。候选越少指标越高，说明隐向量对冷门物品的估计噪声大，把它们排除后排序质量显著提升。最终模型在 maxIter 提高到 15 后验证集为 0.0414（略低于 12 次迭代的 0.0416，差异在小数第四位）。

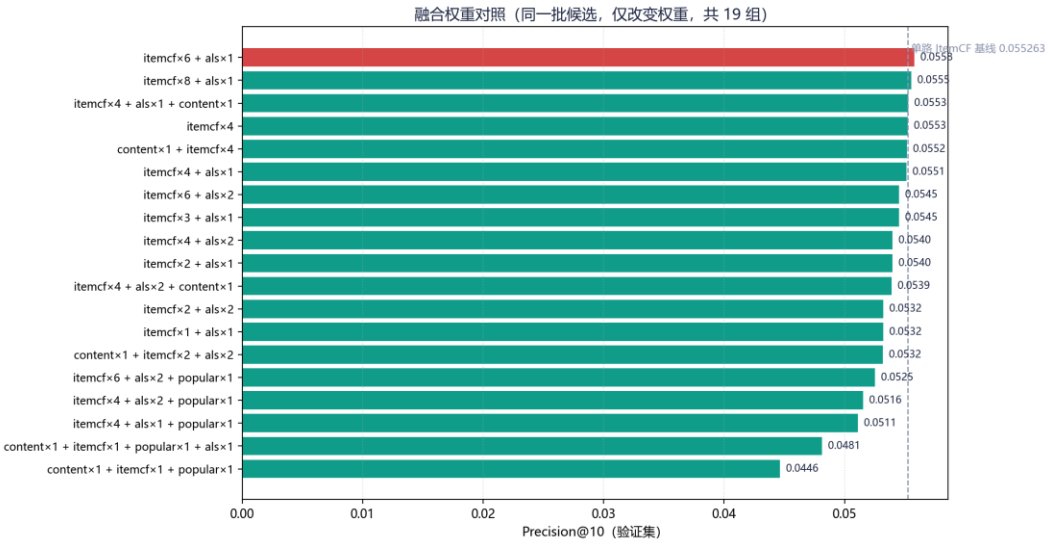


图 6-4 融合权重网格（19 组配置的验证集 Precision@10）

融合权重实验：ItemCF 单路为 0.0553 / 0.0978；ItemCF×6 + ALS×1 为 0.0558 / 0.0974（Precision 略优、Recall 略低）；等权与把 Popular、Content 加入的组合都低于这两个配置，说明权重应当向强路倾斜，弱路只在能补充不同候选时才有价值。测试集上融合为 0.0522，与单路 ItemCF 的 0.0517 接近，说明融合的主要收益是召回覆盖而不是分数平均。

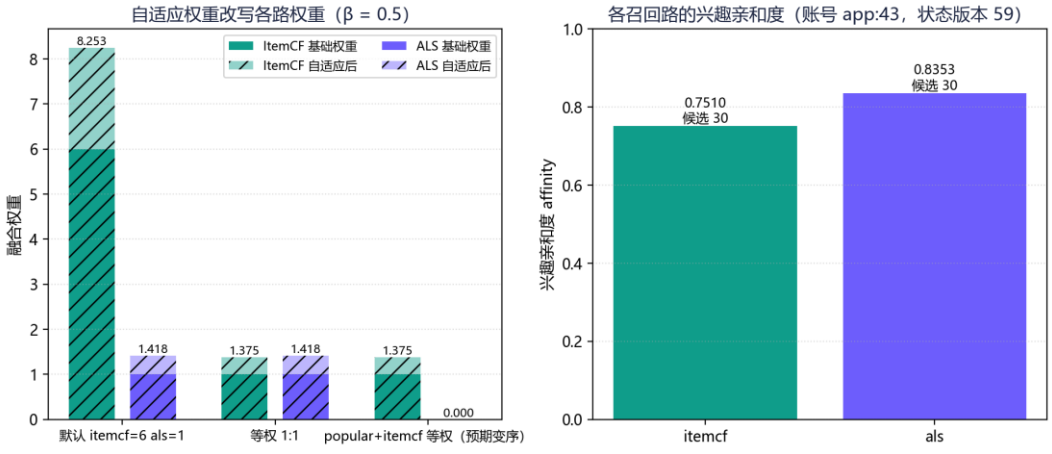


图 6-5 自适应权重的三组对照（基础权重与调整后权重、名次变化）

兴趣与自适应实验：开启短期兴趣后，同一候选集合中匹配度低的电影 356 从第 6 位降到第 9 位，三部高匹配电影各上升一位，说明重排确实生效且方向符合设计；自适应权重在三组预设下分别把 ItemCF 权重从 6.0 调到 8.2529、把 ALS 从 1.0 调到 1.4176（默认预设），在等权预设下 ALS 调整后权重超过 ItemCF 并使名次改变，在 popular+itemcf 预设下 Top-10 顺序从 260,356,1270… 变为 260,1270,356…。三组对照共同说明自适应权重不是只改数字，而是改变排序。

参数实验的限制：所有参数选择都基于单个验证集分区，没有做交叉验证；融合权重网格以 0.5 为步长，未做更细搜索；ALS 的 maxIter 只对比了 12 与 15 两档。这些都可以作为后续改进方向（第 7.2 节）。

6.6 实验结果讨论

6.6.1 方法与事实说明

结果讨论围绕三个问题展开：五类算法在本数据集上的相对表现及其原因；扩展模块（融合、实时兴趣、自适应权重、助手）带来的实际变化；本实验结论的适用边界与效度威胁。讨论中的每一条结论都对应前文给出的表格或脚本报告，不使用未验证的判断。

效度威胁从四个方面检查：数据方面（单次时间划分、单个数据集、电影元数据缺失）、实现方面（容器内推理只用标准库，可能与使用 numpy 的参考实现存在数值细节差异）、协议方面（Top-10 命中率只反映相关物品是否靠前，不反映满意度）、环境方面（本机网络受限导致部分对照实验只能用离线方式完成）。逐条列出威胁比笼统声明“结果可能不准确”更有价值。

表 6-6 6.6 实验结果讨论

项目	记录 1	记录 2	记录 3	记录 4
讨论问题	算法相对表现	扩展带来的变化	适用边界	效度威胁
证据来源	validation / test 指标表	融合与兴趣对照报告	冷启动分区趋势	实验设置与实际限制
结论性质	本协议下的事实	可观测的名次变化	适用范围说明	局限清单
后续动作	多折验证	更细权重搜索	引入元数据	环境与依赖治理

6.6.2 证据与分析

综合结论一：在本数据集的冻结协议下，基于共现的 ItemCF 是性价比最高的方案（验证集 0.0553、测试集 0.0517），矩阵分解 ALS 次之（0.0414 / 0.0396），热门榜是可靠但无个性化的兜底（0.0342 / 0.0312），纯类型特征的内容推荐最弱（0.0041

/ 0.0037），冷启动算法在有少量行为时等价于内容推荐、在无行为时等价于热门榜。这一排序在验证集与测试集上一致，并与冷启动分区的趋势互相印证。

综合结论二：扩展模块的效果是“可观测的名次变化”而不是“指标提升”。融合把验证集 Precision@10 从 0.0553 提升到 0.0558（提升约 0.9%，且 Recall 略降），实时兴趣让低匹配电影下降三位、高匹配电影各上升一位，自适应权重在三组预设下改变权重与顺序。这些变化的共同点是可以被复现（同参数、同数据、同脚本得到同一结果），因此适合作为“系统真的在按兴趣工作”的证据，而不适合宣称“推荐质量显著提升”。

综合结论三：本项目的实现边界清晰。在线推理只用标准库，训练与评价在宿主完成；扩展能力通过独立模块与文件状态实现，不改课程冻结的表结构与接口契约；未完成的模块（公网部署、联网重建离线镜像、元数据补全）已在第 7.2 节列出。这样的取舍让“已验证”与“未验证”泾渭分明，也便于后续继续推进。

总结与展望

7.1 工作总结

本课程设计围绕“把推荐系统从算法做到可用系统”这一目标展开，完成了数据、算法、系统、评价四个层面的工作。数据层面，读取并校验了课程冻结数据包，确认了目录 8,459 部、历史用户 5,000 位、训练 583,502 条、验证 70,366 条、测试 67,792 条与冷启动四个分区的规模，并在实现中严格区分业务数据与评价数据。算法层面，实现了 Popular、Content、ItemCF、ALS 与冷启动五类召回算法，为每个算法生成模型工件与清单，通过课程的模型校验、预测校验与结果校验；各项参数（类型权重、支持度阈值、邻居上限、隐向量维度与正则、候选支持度下限）都在文档与清单中留痕。

系统层面，算法通过统一的推荐契约接入后端与前端：五类算法各有接口与结果卡片，推荐实验室提供数据来源切换、算法对照、融合面板、评价表、行为面板与助手面板；扩展能力包括加权 RRF 融合、Kafka 实时行为与短期兴趣重排、受控工具调用的自然语言助手以及兴趣自适应融合权重；展示层完成了海报缓存、浅色主题与排布优化。评价层面，按冻结协议执行四算法（含冷启动）在两套分区上的评价，完成三项参数对照实验（ALS 维度与正则、候选支持度下限、融合权重）与两项扩展

对照（兴趣重排、自适应权重），并把结果发布到文档与页面。

工程与协作层面，仓库以六次主提交呈现完整演进：环境与基线、实验一、实验二、实验三（含课堂任务补充）、实验四与实验五扩展、实验六挑战版；每次提交都带有主题、范围与验证说明，最终把同一套代码以分支形式推送到小组协作仓库，便于组长与指导教师复核。全组在需求评审、结果复核与文档校对环节共同参与，具体分工见附录 A。

- 完成：五类算法与统一评价协议；融合、实时行为、助手、自适应权重四项扩展；集成检查与场景脚本；文档与截图证据。
- 数据：目录 8,459 部；训练 583,502 条；验证 4,209 人；测试 4,198 人；冷启动每区 499 人。
- 结果：ItemCF 验证集 0.0553 / 0.0978 最优；ALS 0.0414 / 0.0672；热门榜冷启动兜底 0.3369。
- 交付：仓库（主仓库 + 协作仓库分支）、报告、答辩材料与可复现脚本。

7.2 不足与展望

第一项不足是数据层面的：电影元数据中导演、主演、时长、语言、简介与海报字段在数据包中为空，本项目的特征工程因此只能建立在类型标签与交互行为之上。这限制了内容推荐与冷启动的兴趣入口（无法按导演或演员筛选），也使“冷启动引导”只能以类型为粒度。后续改进方向是在合规前提下引入公开元数据源，并保持“真实数据、可追溯来源”的原则，而不是以示例数据填充。

第二项不足是实验层面的：评价只使用课程给定的单次时间划分，没有做多折交叉验证与显著性检验，算法之间在小数第三位的差距无法断言稳定；参数网格的步长较粗（融合权重 0.5、ALS 迭代只比较 12 与 15 两次）；扩展功能的评估以“名次变化可复现”为标准，没有做用户研究或线上 A/B 实验。后续可以补充时间序列多折验证、更细的权重搜索与离线模拟的在线指标（例如覆盖率、多样性与新颖性）。

第三项不足是工程层面的：本机无法访问 Docker Hub，课程文档中的联网重建离线镜像路径未能验证，扩展镜像改用离线安装包与本地派生镜像完成；Kafka 与消费端以单节点、单机方式运行，未验证多分区并行消费与故障恢复；海报缓存依赖外部公开页面，存在来源变动导致失效的风险，需要定期刷新与更完善的兜底策略；自然语言助手依赖外部模型服务，离线环境下不可用（此时按契约返回 503 而不降级造假）。这些限制都已在文档中如实记录，并给出后续改进顺序：先做依赖治理与镜

像可重建，再扩展到多节点与外部数据源。

表总结与展望-1 7.2 不足与展望

项目	记录 1	记录 2	记录 3	记录 4
不足类别	数据	实验	工程	评测
具体表现	元数据字段为空	单次划分、无显著性检验	镜像不可联网重建、单节点 Kafka	只有离线指标，无用户研究
影响	内容与冷启动受限	差距稳定性未知	部署与容错受限	满意度未知
后续计划	引入合规元数据源	多折验证与更细网格	依赖治理与多节点验证	补充覆盖度与多样性指标

参考文献

[1] Koren Y, Bell R, Volinsky C. Matrix factorization techniques for recommender systems[J]. Computer, 2009, 42(8): 30-37.

[2] Sarwar B, Karypis G, Konstan J, et al. Item-based collaborative filtering recommendation algorithms[C]//Proceedings of the 10th International Conference on World Wide Web. New York: ACM, 2001: 285-295.

[3] Hu Y, Koren Y, Volinsky C. Collaborative filtering for implicit feedback datasets[C]//Proceedings of the 8th IEEE International Conference on Data Mining. Piscataway: IEEE, 2008: 263-272.

[4] Rendle S, Freudenthaler C, Gantner Z, et al. BPR: Bayesian personalized ranking from implicit feedback[C]//Proceedings of the 25th Conference on Uncertainty in Artificial Intelligence. Arlington: AUAI Press, 2009: 452-461.

[5] Cremonesi P, Koren Y, Turrin R. Performance of recommender algorithms on top-n recommendation tasks[C]//Proceedings of the 4th ACM Conference on Recommender Systems. New York: ACM, 2010: 39-46.

[6] Cormack G V, Clarke C L A, Buettcher S. Reciprocal rank fusion outperforms condorcet and individual rank learning methods[C]//Proceedings of the 32nd International ACM SIGIR Conference. New York: ACM, 2009: 758-759.

[7] Lops P, de Gemmis M, Semeraro G. Content-based recommender systems: state of the art and trends[M]//Recommender Systems Handbook. Boston: Springer, 2011: 73-105.

[8] Pazzani M J, Billsus D. Content-based recommendation systems[M]//The Adaptive Web. Berlin: Springer, 2007: 325-341.

[9] Harper F M, Konstan J A. The MovieLens datasets: history and context[J]. ACM Transactions on

Interactive Intelligent Systems, 2015, 5(4): 1-19.

[10] Resnick P, Varian H R. Recommender systems[J]. Communications of the ACM, 1997, 40(3): 56-58.

[11] Linden G, Smith B, York J. Amazon.com recommendations: item-to-item collaborative filtering[J]. IEEE Internet Computing, 2003, 7(1): 76-80.

[12] Herlocker J L, Konstan J A, Terveen L G, et al. Evaluating collaborative filtering recommender systems[J]. ACM Transactions on Information Systems, 2004, 22(1): 5-53.

[13] Zhou T, Kuscsik Z, Liu J G, et al. Solving the apparent diversity-accuracy dilemma of recommender systems[J]. Proceedings of the National Academy of Sciences, 2010, 107(10): 4511-4515.

[14] Zaharia M, Xin R S, Wendell P, et al. Apache Spark: a unified engine for big data processing[J]. Communications of the ACM, 2016, 59(11): 56-65.

[15] Kreps J, Narkhede N, Rao J. Kafka: a distributed messaging system for log processing[C]//Proceedings of the NetDB Workshop. 2011: 1-7.

[16] Kleppmann M. Designing data-intensive applications[M]. Sebastopol: O'Reilly Media, 2017.

[17] 项亮. 推荐系统实践[M]. 北京: 人民邮电出版社, 2012.

[18] 李航. 统计学习方法: 第 2 版[M]. 北京: 清华大学出版社, 2019.

[19] 王喆. 深度学习推荐系统[M]. 北京: 电子工业出版社, 2020.

[20] DeepSeek. DeepSeek API documentation[EB/OL]. https://api-docs.deepseek.com, 2025.

附录 A 小组成员分工与贡献

表 A-1 附录 A 小组成员分工与贡献

姓名与学号	成员角色	主要工作与对应提交
韦余圳（学号：［待填］）	组长	总体方案与进度协调；加权 RRF 融合与自适应权重模块；仓库与提交管理（第 4 次、第 6 次提交）
王文华（学号：［待填］）	算法实现	ItemCF 与 ALS 模型构建、离线评价脚本与参数对照实验（第 2 次、第 3 次提交）
杜伟（学号：［待填］）	算法实现	内容推荐与冷启动实现、模型工件与清单维护、评价协议核对（第 1 次、第 2 次提交）
王宇漩（学号：［待填］）	前端与展示	推荐实验室与融合面板、海报缓存与主题改造、逐页验收截图（第 4 次、样式与海报提交）

刘康（学号：［待填］）	数据与测试	Kafka 行为链路与消费端、场景脚本与集成检查、文档整理（第 5 次、第 6 次提交）

附录 B 仓库与提交记录

表 B-1 附录 B 仓库与提交记录

项目	记录
GitHub 仓库（私有）	https://github.com/20041021-hub/cinelab-recommendation-lab
协作仓库分支	https://github.com/weiyuzhen666/-9-（分支 cinelab-student-20041021-hub）
最终提交	1ade9c1（首页冷启动入口），其上为 ccd2a66（海报与浅色主题截图）、8a86023（真实海报接口）、08b77a6、1b732d1、8e633fc、9c7cf82、2ce4187
主提交脉络	bee23a7 基线 → dbe7516 实验一 → d6a0581 实验二 → a22c239/dffcb01/d616d0d 实验三 → d605bf0/8f4c415 实验四 → 8c1cc73/43c7f25 实验五 → 68299d5/2308f73/694f00d 实验六
运行说明	docker compose -f compose.yaml -f deployment/lab5/compose.lab5.yaml -f deployment/lab6/compose.lab6.yaml up -d；访问 http://127.0.0.1:8080
数据版本	dataset rec-course-v3-train5-1.0.0；split temporal-v1-cold500-train5-v3；schema student-eight-v3
主要修改文件	recommender/student/*.py；backend/app/{fusion,interest,behavior,behavior_api,llm,assistant,posters}.py；frontend/src/components/{FusionPanel,AssistantPanel}.vue；scripts/*.py；deployment/lab5、lab6

附录 C 补充实验与测试记录

表 C-1 附录 C 补充实验与测试记录

记录	条件	结果与证据
集成检查	四个容器 healthy；真实账号登录后 31 项断言	31 项全部通过，失败 0（reports/lab6/integration.json）
自然语言助手	正常提问 / 无结果提问 / 非法类型 / 外部服务不可用或缺失密钥	四场景全部通过；失败场景返回 503 且业务接口不受影响（reports/lab6/assistant-scenarios.json）
实时行为	正常写入 / 无历史 / 非法规格 / Kafka 停机	四场景全部通过；停机时业务不中断、失败可见（reports/lab5/scenarios*.json）
自适应权重	默认权重、等权、popular+itemcf 三组预设，adaptive 0/1 对照	三组全部通过；权重与名次变化均有记录（reports/lab6/adaptive-weights.json）
离线评价复现	宿主 Python 执行评价脚本，逐分区输出	五算法 × 六分区结果与文档表格一致（reports/lab3/summary.json）

附录 D AI 辅助开发说明

表 D-1 附录 D AI 辅助开发说明

工具与用途	典型问题	人工验证与修改
AI 编码助手（DeepSeek Harness）：脚手架、脚本与文档初稿	生成的脚本出现路径重复（/api/v1 前缀叠加导致 404）	逐条运行脚本，按真实报错修正并把断言写入集成检查
AI 编码助手：算法实现与调参脚本	热门榜并列排序未按编号升序，违反课程契约	人工对照契约文档，修改排序键并新增回归断言
AI 编码助手：模型清单与工件生成	清单包含中文，评审端按系统编码读取失败	统一为 ASCII 清单，增加构建期检查与文档说明
外部模型服务（DeepSeek API）：系统运行时助手的模型提供方	工具消息缺少 tool_call_id 导致 422	抓取最小复现请求，修正消息结构并重跑四场景脚本
AI 编码助手：前端改造与截图脚本	自动化点击隐藏复选框导致状态与预期相反	改为读取状态后按需点击，并逐张核对截图内容
人工工作：需求确认、结果复核、文档校对、答辩准备	—	由小组成员共同完成；所有结论以仓库中的运行结果与脚本报告为准